MITIGASI HALUSINASI PADA *RAG-LLM* MENGGUNAKAN *QUERY REWRITING, CONTEXT RERANKING* DAN *ACTIVE HALLUCINATION DETECTION*

Laporan Tugas Akhir

Oleh

Ricky Wijaya
18222043

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Maret 2026

LEMBAR PENGESAHAN

MITIGASI HALUSINASI PADA *RAG-LLM* MENGGUNAKAN *QUERY REWRITING, CONTEXT RERANKING* DAN *ACTIVE HALLUCINATION DETECTION*

Laporan Tugas Akhir

Oleh

Ricky Wijaya
18222043

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 28 Maret 2026

Pembimbing

Dr. Fetty Fitriyanti Lubis, S.T., M.T.

NIP. 118110071

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 28 Maret 2026

Ricky Wijaya
NIM: 18222043

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	**Jenis**	**Alat Bantu**	**Tingkat**	**Tempat**
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II hal. 15
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	…	…	…
5	Penyusunan bahan diskusi	…	…	…
6	Penyusunan kode program	…	…	…
7	Penyusunan formula atau perhitungan matematis	…	…	…
8	Penyusunan konsep desain atau algoritma	…	…	…
9	Penyusunan analisis data	…	…	…
10	Penyusunan kesimpulan atau rekomendasi	…	…	…

Bandung, 28 Maret 2026

Ricky Wijaya
NIM: 18222043

ABSTRAK

Implementasi Sistem Smart Scale Berbasis IoT dengan Arsitektur Desentralisasi untuk Optimalisasi Rantai Pasok Jeruk di Tingkat Lapak

oleh

John Doe
123456789

Proses manual dalam rantai pasok jeruk di tingkat lapak, yang mencakup sortir, timbang, dan catat, terbukti tidak efisien, memakan waktu, dan rentan terhadap kesalahan. Meskipun solusi berbasis Internet of Things (IoT) dapat mengatasi masalah ini, arsitektur terpusat konvensional berbasis cloud justru menimbulkan masalah baru berupa latensi tinggi yang menghambat alur kerja real-time. Penelitian ini bertujuan untuk merancang dan mengimplementasikan sebuah purwarupa sistem smart scale berbasis IoT dengan pendekatan desentralisasi (edge computing) untuk meminimalkan latensi dan meningkatkan efisiensi operasional. Metode yang digunakan adalah Model Purwarupa, yang diwujudkan dalam bentuk perangkat keras dengan arsitektur prosesor ganda (Raspberry Pi 5 dan ESP32) yang mampu mengintegrasikan fungsi timbang dan analisis kualitas citra secara simultan. Hasil evaluasi menunjukkan bahwa purwarupa dengan arsitektur edge computing yang diimplementasikan mampu memberikan respons 43,5% lebih cepat (latensi rata-rata 18,05 detik) dibandingkan dengan simulasi arsitektur terpusat. Selain itu, sistem ini berhasil meningkatkan efisiensi waktu kerja secara keseluruhan sebesar 58,32% jika dibandingkan dengan metode manual konvensional. Pengujian Penerimaan Pengguna (UAT) yang melibatkan 30 partisipan juga menunjukkan penerimaan yang sangat positif, dengan skor rata-rata untuk kemudahan penggunaan (5,0/5,0), kejelasan antarmuka (4,87/5,0) dan persepsi kinerja (4,95/5,0). Kesimpulannya, implementasi sistem smart scale dengan arsitektur desentralisasi terbukti merupakan solusi yang valid dan efektif untuk menjawab permasalahan latensi dan inefisiensi di lapak. Sistem ini tidak hanya unggul secara teknis dalam hal performa, tetapi juga fungsional dan dapat diterima dengan baik oleh pengguna akhir, serta berpotensi besar untuk modernisasi rantai pasok agribisnis.

Kata kunci: Smart Scale, Internet of Things, Edge Computing, Rantai Pasok Jeruk.

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul ”Implementasi Algoritma XYZ untuk Optimasi Jaringan Komputer”. Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026
Penulis

John Doe

DAFTAR ISI

LEMBAR PENGESAHAN . . . iii
PERNYATAAN ORISINALITAS . . . v
PERNYATAAN PENGGUNAAN AI . . . vii
ABSTRAK . . . ix
KATA PENGANTAR . . . xi
DAFTAR ISI . . . xiii
DAFTAR GAMBAR . . . xvii
DAFTAR TABEL . . . xix
DAFTAR PERSAMAAN . . . xxi
DAFTAR ALGORITMA . . . xxiii
DAFTAR *LISTING* . . . xxv
DAFTAR SIMBOL . . . xxvii
DAFTAR SINGKATAN . . . xxxi

I PENDAHULUAN . . . 1
I.1 Latar Belakang . . . 1
I.2 Rumusan Masalah . . . 2
I.3 Tujuan . . . 3
I.4 Batasan Masalah . . . 3
I.5 Metodologi Tugas Akhir . . . 4
I.5.1 *Business Understanding* . . . 5
I.5.2 *Data Understanding* . . . 5
I.5.3 *Data Preparation* . . . 5
I.5.4 *Modeling* . . . 6
I.5.5 *Evaluation* . . . 6
I.5.6 *Deployment* . . . 6
I.6 Sistematika Penulisan . . . 6

II STUDI LITERATUR . . . 9
II.1 *Large Language Model* (LLM) . . . 9
II.1.1 *Retrieval-Augmented Generation* . . . 11
II.1.2 *Hallucinations* atau Konfabulasi pada *Large Language Models* . . . 12
II.2 Mitigasi Halusinasi pada *Retrieval-Augmented Generation* . . . 16
II.2.1 Kerangka RAG dan Titik Munculnya *Hallucination* . . . 17
II.2.2 Mitigasi Halusinasi pada Tahap *Retrieval* . . . 18
II.2.2.1 Perbaikan Kueri dan *Query Rewriting* . . . 18
II.2.2.2 Peningkatan Kualitas *Retriever* dan *Reranking* . . . 19
II.2.2.3 Seleksi dan Kompresi Konteks . . . 19
II.2.3 Mitigasi pada Tahap Integrasi Pengetahuan . . . 20
II.2.4 Mitigasi Halusinasi pada Tahap *Generation* . . . 20
II.2.4.1 *Prompt Engineering* dan Strategi Penalaran . . . 20
II.2.4.2 Verifikasi Jawaban dan *Active Detection* . . . 21
II.3 *Query Rewriting* untuk Mitigasi Halusinasi pada RAG . . . 21

II.4 *Reranking* Konteks untuk Mengurangi Kebisingan 23
II.5 *Active Detection* untuk Mitigasi *Hallucination* 24
II.5.1 Prinsip Dasar *Active Detection* 24
II.5.2 Alur Kerja *Active Detection* 25
II.5.3 Kelebihan Metode *Active Detection* 25
II.5.4 Keterbatasan 25
II.6 *Prompt Engineering* untuk Reduksi *Hallucination* 26
II.6.1 *Zero-shot* dan *few-shot prompting* 27
II.7 Kerangka Kerja Evaluasi Sistem *Retrieval-Augmented Generation (RAG)* 28
II.7.1 Aspek Penilaian dalam RAGAS 29
II.7.2 Metrik Turunan dalam RAGAS 29

III ANALISIS MASALAH 31
III.1 Analisis Kondisi Saat Ini 31
III.2 Analisis Kebutuhan 35
III.3 Analisis Pemilihan Solusi 37
III.3.1 Alternatif Solusi 37
III.3.1.1 Alternatif Solusi I: Penguatan Tahap *Retrieval* (*Query Rewriting*, *Query Expansion*, dan *Embedding*) . 37
III.3.1.2 Alternatif Solusi II: Penguatan Tahap *Generation* melalui *Middle Information*, *Context Reranking*, dan *Balance Internal and External Knowledge* 39
III.3.1.3 Alternatif Solusi III: *Multiple-Phase Mitigation* (*Query Rewriting*, *Context Reranking*, dan *Active Hallucination Detection*) 40
III.3.2 Analisis Penentuan Solusi 41

IV DESAIN SOLUSI 45
IV.1 Tahapan Desain 45
IV.1.1 Persiapan Data (*Data Preparation*) 45
IV.1.2 Pemodelan (*Modeling*) 46
IV.1.2.1 Konfigurasi Sistem 46
IV.1.2.2 Desain *Pipeline* RAG *Baseline* 47
IV.1.2.3 Desain Komponen Mitigasi Halusinasi 48
IV.2 Hasil Desain 50
IV.2.1 Hasil Persiapan Data 50
IV.2.2 Hasil Pemodelan 51
IV.2.2.1 Hasil *Pipeline* RAG *Baseline* 51
IV.2.2.2 Hasil Komponen Mitigasi Halusinasi 51
IV.2.2.3 Hasil Implementasi *Pipeline* Terpadu 52

V IMPLEMENTASI 55
V.1 Lingkungan Implementasi 55
V.1.1 Perangkat Keras 55
V.1.2 Perangkat Lunak dan Dependensi 55
V.2 Implementasi Persiapan Data 56

V.3 Implementasi *Pipeline* RAG *Baseline* 57
V.3.1 Implementasi *Retrieval Pipeline* 57
V.3.2 Implementasi *Generation Pipeline* 57
V.3.3 Implementasi Penyimpanan Hasil Inkremental 57
V.4 Implementasi Komponen Mitigasi Halusinasi 58
V.4.1 Implementasi *Query Rewriting* 58
V.4.2 Implementasi *Context Reranking* 58
V.4.3 Implementasi *Active Hallucination Detection* 58
V.5 Implementasi *Framework* Evaluasi 58

VI EVALUASI **61**
VI.1 Metodologi Evaluasi 61
VI.1.1 Metrik Evaluasi RAGAS 61
VI.1.2 Metrik Evaluasi Tambahan 63
VI.1.3 Prosedur Evaluasi 63
VI.2 Hasil Evaluasi 63
VI.2.1 Hasil Konfigurasi *Baseline* 63
VI.2.2 Hasil Konfigurasi *Baseline* + *Query Rewriting* 64
VI.2.3 Hasil Konfigurasi *Baseline* + *Context Reranking* 64
VI.2.4 Hasil Konfigurasi *Baseline* + *Active Hallucination Detection* 65
VI.2.5 Hasil Konfigurasi *Combined* 65
VI.3 Perbandingan Antar Konfigurasi 65
VI.4 Analisis dan Pembahasan 66
VI.4.1 Dampak *Query Rewriting* 66
VI.4.2 Dampak *Context Reranking* 66
VI.4.3 Dampak *Active Hallucination Detection* 66
VI.4.4 Analisis Konfigurasi *Combined* 66
VI.4.5 Diskusi Umum 67

VII PENUTUP **69**
VII.1 Kesimpulan 69
VII.2 Saran 69

Lampiran A KODE PROGRAM **71**
A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik . . . 71
A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak 72

Lampiran B HASIL SURVEI LAPANGAN **73**
B.1 Foto Survei Lokasi 1 73
B.2 Foto Survei Lokasi 2 73
B.3 Transkrip Wawancara dengan Narasumber 73

DAFTAR GAMBAR

I.1 Tahapan Metodologi CRISP-DM (**crispdm_datasciencepm**) 5

II.1 Alur *Retrieval-Augmented Generation* (**gao2023rag**) 12

II.2 Penyebab dan solusi halusinasi pada RAG-LLM (**math2024hallucination<empty citation>**

III.1 Alur sistem *retrieval augmented generation* (RAG) saat ini 32

DAFTAR TABEL

II.1 Contoh berbagai jenis *hallucination* pada *Large Language Models (LLM)* 13

III.1 Perbandingan Dataset Kandidat 32
III.2 Kebutuhan Fungsional Sistem RAG-LLM untuk Mitigasi Halusinasi 36
III.3 Kebutuhan Nonfungsional Sistem RAG-LLM untuk Mitigasi Halusinasi 37
III.4 Kalkulasi *weighted scoring* pada alternatif solusi mitigasi *hallucination* 43

IV.1 Matriks Konfigurasi Sistem RAG 46
IV.2 Parameter Teknis Sistem RAG 47
IV.3 Statistik Dataset PubMedQA *pqa_labeled* 50
IV.4 Contoh Entri Dataset PubMedQA Setelah Prapemrosesan 51
IV.5 Contoh Keluaran *Pipeline* RAG *Baseline* 51
IV.6 Contoh Perbandingan Keluaran Lima Konfigurasi untuk Satu Pertanyaan Sampel 53

V.1 Spesifikasi Perangkat Keras 55
V.2 Daftar Dependensi Utama 56
V.3 Struktur *Notebook* Eksperimen 56

VI.1 Ringkasan Hasil Evaluasi Konfigurasi *Baseline* 64
VI.2 Ringkasan Hasil Evaluasi Konfigurasi *Baseline* + *Query Rewriting* . 64
VI.3 Ringkasan Hasil Evaluasi Konfigurasi *Baseline* + *Context Reranking* 64
VI.4 Ringkasan Hasil Evaluasi Konfigurasi *Baseline* + *Active Hallucination Detection* 65
VI.5 Ringkasan Hasil Evaluasi Konfigurasi *Combined* 65
VI.6 Perbandingan Hasil Evaluasi Seluruh Konfigurasi 66

DAFTAR PERSAMAAN

VI.1 Formula *faithfulness* 61
VI.2 Formula *answer relevancy* 62
VI.3 Formula *context precision* 62
VI.4 Formula *context recall* 62
VI.5 Formula *label accuracy* 63
VI.6 Formula *hallucination rate* 63

DAFTAR ALGORITMA

DAFTAR *LISTING*

A.1 Binary search code 71

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
α	Koefisien bobot untuk *loss function*, tingkat signifikansi statistik	75
β	Koefisien bobot untuk *Binary Cross-Entropy loss*	75
λ	Koefisien bobot untuk CTC *loss*, parameter regularisasi	75
θ	Parameter model *neural network*	75
σ	Deviasi standar	75
μ	Rata-rata (*mean*)	75
ϵ	*Token blank*/kosong dalam CTC, konstanta kecil untuk stabilitas numerik	75
Δ	Delta (selisih/perubahan nilai)	75
π	*Alignment path* dalam CTC	75
π_t	*Token* pada *timestep* t dalam *alignment path*	76
∇ atau ∂	Operator gradien atau turunan parsial	75
$\sum$	Operator penjumlahan	75
$\prod$	Operator perkalian	75
$\int$	Operator integral	75
argmax	Argumen yang memaksimalkan fungsi	76
$\sim$	Terdistribusi menurut	75
$\rightarrow$	Menuju atau konvergen ke	75
$\mathcal{L}$	Fungsi *loss* (kerugian)	75
$\mathcal{L}_{total}$	Total *loss function*	75
$\mathcal{L}_{adversarial}$ atau $\mathcal{L}_{adv}$	*Adversarial loss*	75
$\mathcal{L}_{reconstruction}$	*Reconstruction loss*	75
$\mathcal{L}_{L1}$	L1 *loss* (*Mean Absolute Error*)	75
$\mathcal{L}_{L2}$	L2 *loss* (*Mean Squared Error*)	75
$\mathcal{L}_{CTC}$	CTC (*Connectionist Temporal Classification*) *loss*	75
$\mathcal{L}_{perceptual}$	*Perceptual loss*	75

$\mathcal{L}_{pixel}$	*Pixel-wise loss*	75
$\mathcal{L}_{BCE}$	*Binary Cross-Entropy loss*	75
$\mathcal{L}_{GAN}$	GAN *loss*	75
$\mathcal{L}_{cycle}$	*Cycle consistency loss*	75
G	*Generator* dalam GAN	75
D	*Discriminator* dalam GAN	75
$G(z)$	*Output* dari *generator* dengan *input* z	75
$D(x)$	*Output* dari *discriminator* dengan *input* x	75
$G_{A \to B}$	*Generator* dari domain A ke B	75
$G_{B \to A}$	*Generator* dari domain B ke A	75
$V(D, G)$	Fungsi nilai (*value function*) dalam GAN	75
x	Sampel data asli, *input* citra	75
y	Label atau *ground truth*, kondisi dalam *conditional GAN*	75
z	*Noise vector* atau representasi laten	
$\mathbb{E}$	Ekspektasi atau nilai harapan	75
p_{data}	Distribusi data asli	75
p_z	Distribusi *prior* (*noise*)	75
$p(y\|x)$	Probabilitas bersyarat *output* y *given input* x	75
$p_t(\pi_t\|x)$	Probabilitas *token* pada *timestep* t	
$\mathcal{B}$	Fungsi penciutan (*collapse function*) dalam CTC	75
$\mathcal{B}^{-1}$	Fungsi invers penciutan dalam CTC	75
T	Jumlah *timestep* atau panjang *sequence*	
$\|x - G(y)\|_1$	L1 *distance* antara *ground truth* dan *generated image*	75
$\|x - G(y)\|_2$	L2 *distance* antara *ground truth* dan *generated image*	75
$N \times N$	Ukuran *patch* dalam PatchGAN	
d	Cohen's d (*effect size*)	75
n	Jumlah sampel	75
p	*p-value* (nilai probabilitas statistik)	75

r	Koefisien korelasi	75
R^2	Koefisien determinasi	
$O(n)$	Notasi Big-O untuk kompleksitas linear	75
$O(n^2)$	Notasi Big-O untuk kompleksitas kuadratik	75

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	*Artificial Intelligence*	1
CNN	*Convolutional Neural Network*	2
GPU	*Graphics Processing Unit*	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Kemajuan *Large Language Models (LLM)* seperti *GPT*, *LLaMA*, dan *PaLM* telah menghadirkan perubahan fundamental dalam berbagai aspek kehidupan modern. Model-model ini mampu menghasilkan teks yang koheren, mengikuti instruksi, serta membantu proses analisis informasi secara cepat dan efisien. Hal tersebut membuat LLM semakin banyak digunakan dalam layanan publik, pendidikan, kesehatan, bisnis, hingga penyediaan informasi sehari-hari (**math2024hallucination**).

Namun, di balik kemajuan tersebut, terdapat tantangan serius berupa fenomena *hallucination*. *Hallucination* terjadi ketika model menghasilkan informasi yang tampak meyakinkan tetapi sebenarnya tidak benar, tidak berdasar, atau sepenuhnya fiktif (**lettucedetect2024**). Risiko tersebut bukan sekadar isu teknis karena dampaknya telah terlihat nyata dalam berbagai kasus yang merugikan masyarakat.

Pada tahun 2023, seorang pengacara di New York mengajukan enam rujukan kasus hukum palsu yang sepenuhnya dihasilkan oleh *ChatGPT*, sehingga ia dikenai sanksi pengadilan dan menanggung kerugian reputasi (**schwartz2023chatgptlawcase**). Dalam bidang kesehatan, studi MedPaLM oleh Singhal dkk. menunjukkan bahwa LLM kerap menghasilkan rekomendasi medis yang tidak akurat dan berpotensi membahayakan pasien jika digunakan tanpa verifikasi profesional (**singhal2023medpalm**). Laporan UNESCO (2023) juga menyoroti kemampuan model generatif dalam mempercepat penyebaran misinformasi sehingga berpotensi mengganggu stabilitas informasi publik (**unesco2023misinformation**). Bahkan pada tahun 2024, maskapai Air Canada dinyatakan bertanggung jawab oleh pengadilan karena *chatbot* resminya memberikan informasi pengembalian dana yang keliru, menunjukkan bahwa *hallucination* memiliki konsekuensi hukum dan finansial yang nyata (**aircanada2024refundcase**).

Berbagai insiden tersebut menggambarkan bahwa *hallucination* bukan hanya sekadar kesalahan keluaran model, melainkan isu kritis yang memengaruhi kepercayaan,

keamanan, dan reliabilitas teknologi berbasis LLM. Oleh karena itu, perlu dilakukan upaya sistematis untuk memahami dan memitigasi *hallucination*, khususnya pada pendekatan *Retrieval-Augmented Generation* (RAG) yang menggabungkan kemampuan generatif dengan sumber informasi eksternal.

Secara teoretis, RAG dapat mengurangi *hallucination* karena model menjawab berdasarkan dokumen yang dapat diverifikasi. Namun dalam praktiknya, RAG tetap dapat menghasilkan informasi keliru apabila dokumen yang diperoleh tidak relevan, ambigu, atau tidak sesuai dengan maksud pertanyaan. Dengan demikian, kualitas jawaban tidak hanya bergantung pada kemampuan model generatif, tetapi juga pada proses pencarian dan pemilihan konteks yang tepat.

Tinjauan terbaru oleh Zhang (2025) berjudul *"Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review"* memetakan berbagai penyebab *hallucination* dalam RAG serta merangkum strategi mitigasinya (**math2024hallucination**). Meskipun memberikan gambaran yang komprehensif, tinjauan tersebut menunjukkan bahwa solusi yang ada saat ini masih berdiri sendiri dan belum digabungkan dalam satu sistem terpadu.

Sebagian besar studi hanya menerapkan satu jenis mitigasi pada satu tahap tertentu. Akibatnya, belum terdapat implementasi yang mengintegrasikan beberapa teknik mitigasi dalam satu alur menyeluruh untuk menekan *hallucination* secara lebih efektif. Kesenjangan ini menjadi penting untuk diatasi, terutama pada bidang yang membutuhkan tingkat kebenaran informasi yang tinggi, seperti layanan publik, kesehatan, dan penegakan hukum.

I.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, tugas akhir ini berfokus untuk menjawab beberapa pertanyaan utama sebagai berikut:

1. Bagaimana kinerja sistem RAG standar dibandingkan dengan RAG yang dilengkapi *query rewriting*, *context reranking*, serta *active hallucination detection*?
2. Sejauh mana setiap teknik mitigasi mampu menurunkan tingkat *hallucination*, khususnya untuk kategori *intrinsic hallucination* dan *factual hallucination*?
3. Bagaimana evaluasi kuantitatif dapat digunakan untuk mengukur efektivitas mitigasi menggunakan metrik seperti *hallucination rate*, *accuracy*, *precision*, *recall*, *F1-score*, serta tingkat penurunan *hallucination*?

I.3 Tujuan

Tujuan dari tugas akhir ini adalah sebagai berikut:

1. Menerapkan tiga pendekatan mitigasi yaitu *query rewriting*, *context reranking*, dan *active hallucination detection* pada sistem RAG.
2. Membandingkan kinerja beberapa konfigurasi sistem: RAG standar, *RAG + Query Rewriting*, *RAG + Context Reranking*, *RAG + Active Detection*, serta gabungan seluruh teknik mitigasi.
3. Mengevaluasi dampak mitigasi berdasarkan penurunan tingkat *hallucination* serta skor kinerja model menggunakan metrik kuantitatif seperti *accuracy*, *precision*, *recall*, dan *F1-score*.

I.4 Batasan Masalah

Untuk menjaga fokus dan kejelasan ruang lingkup, tugas akhir ini dibatasi pada analisis dan pengembangan *Large Language Models (LLM)* yang diperkaya dengan mekanisme *Retrieval-Augmented Generation (RAG)*. Tugas akhir ini hanya menyoroti dua jenis *hallucination* yang paling relevan dengan proses *retrieval* dan *generation*, yaitu *halusinasi intrinsik* dan *halusinasi faktual*. *Halusinasi intrinsik* merujuk pada keluaran model yang bertentangan dengan informasi yang terdapat dalam *prompt* atau konteks yang diberikan, sedangkan *halusinasi faktual* berkaitan dengan keluaran yang mengandung informasi keliru atau tidak akurat meskipun tampak meyakinkan. Jenis *hallucination* lain seperti *reasoning hallucination*, *linguistic hallucination*, dan *bias-related hallucination* berada di luar cakupan studi ini.

Dari sisi teknik mitigasi, tugas akhir ini terbatas pada penerapan tiga pendekatan utama, yaitu *query rewriting* untuk memperbaiki kualitas kueri pada tahap *retrieval*, *context reranking* untuk memilih dan menata ulang konteks yang paling relevan, serta *active hallucination detection* untuk mendeteksi keluaran dengan risiko halusinatif pada tahap generasi. Metode mitigasi lain seperti *prompt engineering* lanjutan, *knowledge editing*, atau *model fine-tuning* tidak dibahas dalam tugas akhir ini.

Untuk menjaga konsistensi evaluasi, tugas akhir ini hanya menggunakan metrik kuantitatif yang umum dipakai dalam tugas klasifikasi teks, yaitu *accuracy*, *precision*, *recall*, *F1-score*, evaluasi *RAGAS*, serta tingkat pengurangan *hallucination*.

Selain itu, tugas akhir ini juga dibatasi pada penggunaan data berbahasa Inggris. Seluruh proses *retrieval*, pembentukan konteks, dan evaluasi keluaran model dilakukan menggunakan korpus teks dalam bahasa Inggris. Penelitian tidak mencakup analisis pada bahasa lain di luar ruang lingkup tersebut.

Dari sisi sumber data, penelitian ini hanya menggunakan dataset *PubMedQA* sebagai himpunan data uji. *PubMedQA* dipilih karena menyediakan pasangan pertanyaan, abstrak ilmiah, serta jawaban faktual yang berkaitan langsung dengan konteks penelitian dalam lingkungan medis. Dataset pada domain lain tidak termasuk dalam cakupan studi ini.

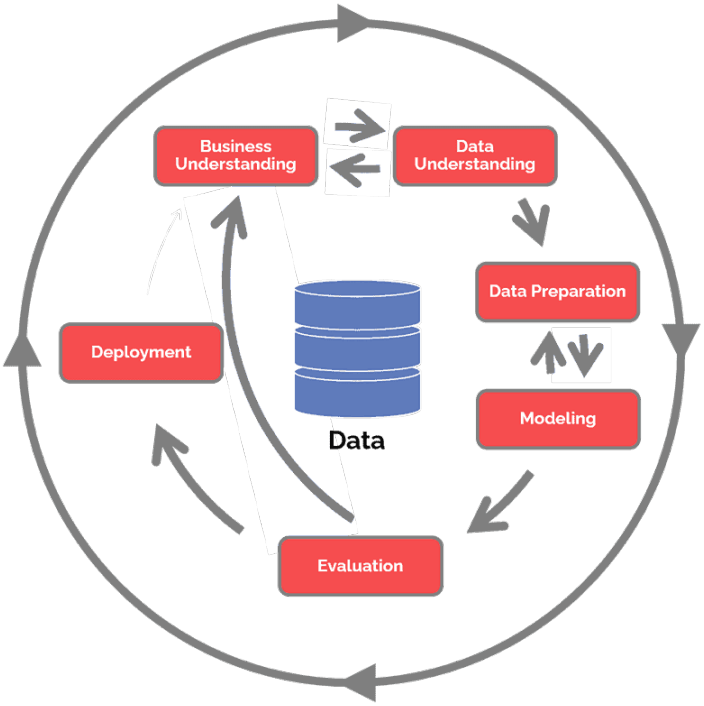
Secara keseluruhan, batasan-batasan tersebut ditetapkan untuk menjaga ruang lingkup penelitian tetap fokus, terukur, dan relevan terhadap tujuan tugas akhir.

I.5 Metodologi Tugas Akhir

Tugas akhir ini menggunakan pendekatan metodologi *Cross-Industry Standard Process for Data Mining* atau disingkat **CRISP-DM**. Metodologi ini merupakan kerangka kerja industri yang banyak digunakan dalam proyek berbasis *data mining* dan *machine learning* karena bersifat fleksibel, iteratif, serta berorientasi pada kebutuhan bisnis dan teknis (**crispdm_datasciencepm**).

CRISP-DM menyediakan tahapan sistematis untuk memahami permasalahan, menyiapkan data, membangun model, hingga mengevaluasi serta mengimplementasikannya. Metodologi ini terdiri atas enam fase utama, yaitu: *business understanding*, *data understanding*, *data preparation*, *modeling*, *evaluation*, serta *deployment*.

Visualisasi tahapan metodologi *CRISP-DM* dapat dilihat pada Gambar I.1.



Gambar I.1 Tahapan Metodologi CRISP-DM (**crispdm_datasciencepm**)

I.5.1 *Business Understanding*

Tahap pertama bertujuan memahami kebutuhan dan tujuan sistem dari sudut pandang pengguna atau masalah operasional. Kegiatan dilakukan untuk menentukan ruang lingkup, sasaran, serta kriteria keberhasilan model yang akan dibangun. Pemahaman konteks pada fase ini menjadi fondasi dalam pemilihan strategi analisis dan pendekatan pemodelan.

I.5.2 *Data Understanding*

Tahap ini berfokus pada eksplorasi data awal, termasuk identifikasi sumber data, penilaian kualitas, serta analisis karakteristik yang relevan dengan permasalahan. Tujuannya adalah memperoleh wawasan mengenai struktur, konten, serta permasalahan yang mungkin muncul pada tahap pemrosesan.

I.5.3 *Data Preparation*

Tahap ini mencakup pembersihan data, seleksi variabel, penggabungan data, transformasi format, serta penanganan *missing values*. Hasil akhir tahap ini adalah dataset terstruktur dengan kualitas memadai untuk digunakan pada proses pemodelan.

I.5.4 *Modeling*

Pada tahap ini ditentukan teknik pemodelan, pemilihan algoritma, serta konfigurasi parameter (*hyperparameter*) yang sesuai. Data yang telah dipersiapkan kemudian digunakan untuk membangun model, termasuk pengujian alternatif parameter untuk mendapatkan kinerja yang optimal.

I.5.5 *Evaluation*

Tahap ini mengevaluasi kinerja model berdasarkan metrik evaluasi kuantitatif. Selain itu, dilakukan pula pengecekan kesesuaian model dengan tujuan tugas akhir. Apabila model belum memenuhi kriteria keberhasilan, diperlukan iterasi kembali pada tahap pemahaman data atau pemodelan.

I.5.6 *Deployment*

Tahap terakhir adalah penerapan model ke dalam lingkungan operasional atau simulasi implementasi. Pada tahap ini dilakukan pemantauan performa, dokumentasi, serta pengelolaan perubahan data dan model agar solusi tetap andal dalam jangka panjang.

Metodologi CRISP-DM menyediakan kerangka terstruktur yang menyeimbangkan kebutuhan teknis dan praktis sehingga tugas akhir berbasis data dapat dijalankan secara sistematis, transparan, dan dapat direproduksi dengan baik.

I.6 Sistematika Penulisan

Tuliskan gambaran umum isi dari setiap bab yang ada dalam laporan tugas akhir. Sistematika penulisan ini memberikan panduan bagi pembaca tentang struktur dan alur pembahasan dalam laporan tugas akhir. Berikut adalah contoh sistematika penulisan yang dapat diikuti:

1. Bab I Pendahuluan: Berisi latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika penulisan laporan tugas akhir .
2. Bab II Studi Literatur: Berisi kajian pustaka yang relevan dengan topik tugas akhir, termasuk teori-teori, konsep-konsep, dan hasil-hasil penelitian sebelumnya yang mendukung pelaksanaan tugas akhir.
3. Bab III Analisis: Berisi penjelasan tentang analisis masalah dan kebutuhan pengguna serta berbagai alternatif solusinya.
4. Bab IV Perancangan: Berisi penjelasan tentang perancangan solusi yang diusulkan, misalnya arsitektur sistem, desain basis data, antarmuka pengguna, dan sebagainya.
5. Bab V Implementasi: Berisi penjelasan tentang proses implementasi solusi

yang diusulkan, termasuk langkah-langkah yang diambil, teknologi yang digunakan, dan hasil yang diperoleh.

6. Bab VI Evaluasi: Berisi analisis dan evaluasi terhadap solusi yang diimplementasikan, termasuk metode pengujian, hal-hal yang disiapkan sebelum pengujian, hasil pengujian, dan penilaian kinerja.
7. Bab VII Kesimpulan dan Saran: Berisi kesimpulan dari hasil pelaksanaan tugas akhir dan saran untuk pengembangan lebih lanjut.
8. Daftar Pustaka: Berisi daftar referensi yang digunakan dalam penyusunan laporan tugas akhir.
9. Lampiran: Berisi dokumen-dokumen pendukung seperti kode program, data lengkap hasil pengujian, rekaman wawancara, dan lain-lain.

BAB II

STUDI LITERATUR

II.1 *Large Language Model* (LLM)

Large Language Model (LLM) adalah keluarga model bahasa berbasis transformer yang dilatih pada korpus teks berskala sangat besar sehingga mampu mempelajari pola bahasa dan memecahkan beragam tugas generatif maupun pemahaman tanpa pelatihan ulang khusus pada tiap tugas (**zhao2023survey**); (**minaee2024large**). Berbeda dari model bahasa generasi awal yang terutama memodelkan distribusi kata, LLM modern menunjukkan kemampuan yang lebih umum untuk menyelesaikan instruksi, menalar, dan beradaptasi pada konteks baru. Kapabilitas kunci LLM yang banyak didokumentasikan meliputi :

1. Pembelajaran dalam konteks (*in-context learning*), yakni ketika LLM mempelajari tugas baru dari sekumpulan contoh yang disajikan dalam *prompt*.
2. Mengikuti instruksi, yakni ketika sudah dilakukan *instruction tuning*, LLM dapat mengikuti instruksi untuk variasi tugas yang baru tanpa perlu ditunjukkan lagi contohnya.
3. *Multi-step reasoning*, yakni ketika LLM dapat menyelesaikan tugas yang kompleks dengan memecah tugas tersebut menjadi langkah-langkah penalaran (**minaee2024large**).

Tonggak penting dari perkembangan LLM adalah kemunculan ChatGPT (*Chat Generative Pretrained Transformer*) pada tanggal 30 November 2022 (**minaee2024large**). ChatGPT merupakan *chatbot* yang dikembangkan oleh OpenAI yang memungkinkan pengguna untuk menyelesaikan berbagai macam tugas, seperti tanya jawab (*question answering*), pencarian informasi (*information seeking*), perangkuman teks, dan banyak lagi (**minaee2024large**). Saat ini ChatGPT didukung oleh model GPT-5 yang merupakan model terbaru dan terkuat dari varian model GPT (**minaee2024large**).

Selain OpenAI, Meta dan Google pun ikut berlomba dalam pengembangan model LLM. Meta mengembangkan model bernama LLaMa yang, tidak seperti model GPT, bersifat *open-source* (**minaee2024large**). Karena sifatnya yang *open-source*,

LLaMa banyak digunakan oleh grup riset dengan tujuan untuk mengembangkan model *LLM open-source* yang dapat berkompetisi dengan model *closed-source* ataupun mengembangkan model LLM pada tugas-tugas tertentu (**minaee2024large**). Dari variasi LLaMa, LLaMa-2, LLaMa-3, hingga LLaMa-3.1, pembaruan serta pengembangan terus dilakukan (**zhao2023survey**). Dengan parameter yang meningkat (versi terbesarnya memiliki 405 miliar parameter), *pre-training tokens* yang lebih banyak (15 triliun token), serta *context window* yang diperluas (128 ribu), model LLaMa-3.1 telah meningkatkan kemampuannya secara signifikan dan mencapai kinerja yang kompetitif dibandingkan model-model *closed-source* lainnya seperti GPT-4, GPT-4o, dan Claude 3.5 Sonnet pada berbagai patokan tertentu (**zhao2023survey**).

Sementara itu, Google mengembangkan model bernama PaLM (*Pathways Language Model*) yang pertama kali diumumkan pada bulan April 2022 (**minaee2024large**). Model ini merupakan model dengan 540 miliar parameter dan 780 miliar token yang mampu mengerjakan berbagai tugas bahasa alami dan *use case* (**zhao2023survey**). Model generasi berikutnya, yakni PaLM-2, merupakan model yang efisien dalam melakukan komputasi dengan kemampuan multibahasa dan penalaran yang lebih baik dibandingkan generasi sebelumnya (**minaee2024large**). Melalui evaluasi yang ekstensif, model PaLM-2 meningkatkan kinerjanya pada berbagai ukuran model dengan tetap menunjukkan inferensi yang lebih cepat dan lebih efisien daripada PaLM (**minaee2024large**).

Walaupun LLM telah mencapai kemajuan dalam mengambil serta memanfaatkan informasi pengetahuan, model ini masih memiliki dua masalah utama (**zhao2023survey**):

1. ***Hallucination*** Salah satu isu yang menantang dalam membuat teks faktual adalah halusinasi, yakni ketika informasi yang dihasilkan oleh model LLM bertentangan dengan sumber yang ada atau tidak dapet diverifikasi oleh sumber yang tersedia (**zhao2023survey**). Halusinasi dapat memengaruhi LLM untuk menghasilkan keluaran yang tidak diinginkan dan dapat menurunkan kinerjanya sehingga hal ini dapat menimbulkan risiko saat diterapkan dalam aplikasi dunia nyata (**zhao2023survey**).
2. ***Domain Knowledge*** Isu lainnya yang dihadapi oleh LLM adalah kesulitan dalam menyelesaikan masalah yang memerlukan pengetahuan terbaru yang ada di luar data pelatihan (**zhao2023survey**). Pendekatan yang dapat dilakukan untuk menghindari isu ini adalah dengan melakukan pembaruan pengetahuan pada LLM dengan data baru (**zhao2023survey**). Namun, pembaruan pengetahuan dengan data baru perlu dilakukan secara efisien dan efektif untuk menghindari biaya yang mahal dalam melakukan *fine tuning* serta menghin-

dari masalah kelupaan yang fatal pada LLM (**zhao2023survey**).

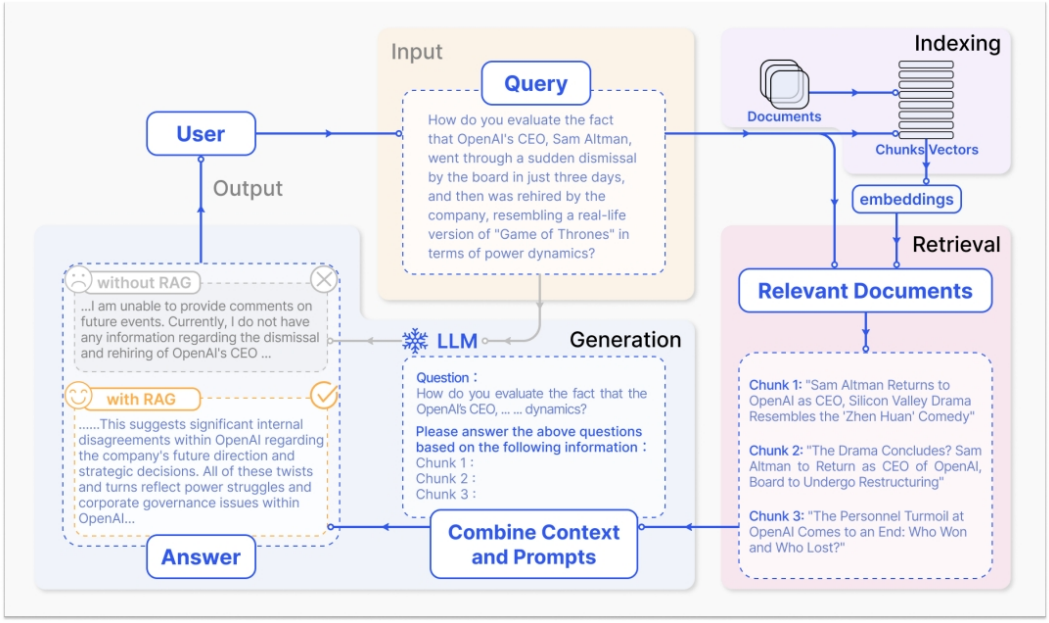
II.1.1 *Retrieval-Augmented Generation*

Retrieval-Augmented Generation (RAG) merupakan pendekatan yang memperkuat kemampuan *Large Language Model* (LLM) dengan mengakses potongan dokumen yang relevan dari sumber pengetahuan eksternal melalui perhitungan kesamaan semantik (**gao2023rag**). Dengan merujuk pada basis pengetahuan tersebut, RAG mampu mengurangi kecenderungan model menghasilkan konten yang salah secara faktual karena jawaban yang diberikan diarahkan pada bukti tekstual yang dapat diverifikasi.

Secara umum, mekanisme RAG terdiri atas tiga tahapan utama yang saling berkaitan, yaitu *indexing*, *retrieval*, dan *generation*. Pada tahap *indexing*, sistem melakukan ekstraksi serta pembersihan data dari berbagai format dokumen seperti PDF, HTML, Word, maupun Markdown. Seluruh data tersebut dikonversi menjadi teks murni dan kemudian dipecah menjadi segmen-segmen kecil (*chunks*) yang sesuai dengan batas kapasitas konteks model bahasa. Setiap segmen diubah menjadi representasi vektor menggunakan model *embedding*, lalu disimpan ke dalam basis data vektor untuk memungkinkan pencarian berbasis kesamaan secara efisien.

Tahap berikutnya, yaitu *retrieval*, dimulai ketika pengguna memasukkan kueri. Sistem mengonversi kueri tersebut menjadi vektor menggunakan model *embedding* yang sama seperti pada proses indeks. Kueri dalam bentuk vektor kemudian dibandingkan dengan vektor segmen dalam basis data, sehingga sistem dapat mengambil beberapa segmen dengan skor kesamaan tertinggi. Hasil pencarian inilah yang menjadi konteks pengetahuan bagi model.

Tahap terakhir adalah *generation*, di mana kueri dan konteks terpilih digabungkan menjadi sebuah *prompt* yang kemudian diberikan kepada LLM. Model menghasilkan jawaban sesuai tugas yang diberikan, dan pada beberapa kasus, riwayat percakapan dapat digabungkan sehingga mendukung interaksi multi-putaran (*multi-turn dialog*). Dengan cara ini, proses generasi menjadi lebih terarah dan berbasis bukti, mengurangi risiko *hallucination* serta meningkatkan akurasi faktual (**gao2023rag**).



Gambar II.1 Alur *Retrieval-Augmented Generation* (**gao2023rag**)

II.1.2 *Hallucinations* atau Konfabulasi pada *Large Language Models*

Konsep *hallucination* pada awalnya berasal dari bidang psikologi, yang mengacu pada pengalaman perseptual tanpa adanya rangsangan eksternal atau persepsi yang menyimpang dari realitas. Dalam konteks *Natural Language Processing* (NLP), halusinasi merujuk pada keluaran model bahasa besar (*Large Language Model* atau LLM) yang tidak konsisten dengan fakta atau bahkan tidak masuk akal. Namun, beberapa peneliti menilai bahwa istilah *hallucination* tidak sepenuhnya tepat karena memberikan kesan bahwa model memiliki persepsi sensorik sebagaimana manusia. Sebagai alternatif, istilah *confabulation* dianggap lebih akurat, karena menggambarkan kecenderungan seseorang untuk ”mengisi celah memori” dengan informasi yang tampak benar namun sebenarnya keliru, suatu analogi yang sesuai dengan perilaku LLM ketika menghasilkan keluaran yang salah namun tampak meyakinkan (**math2024hallucination**).

Dalam praktiknya, istilah *hallucination* masih lebih banyak digunakan dalam literatur akademik karena telah menjadi istilah baku untuk menggambarkan fenomena keluaran tidak faktual pada LLM. Secara umum, halusinasi dapat dibedakan menjadi dua kategori utama, yaitu:

a. **Halusinasi intrinsik**, yaitu ketika keluaran model bertentangan dengan masukan atau konteks sumber. Contohnya, model memberikan angka yang berbeda dari data yang sebenarnya terdapat dalam teks masukan.

b. **Halusinasi ekstrinsik**, yaitu ketika keluaran tidak dapat diverifikasi dari masukan namun mungkin benar jika dibandingkan dengan pengetahuan eksternal (**math2024hallucination**).

Klasifikasi yang lebih mendalam dikemukakan oleh Huang et al. (2023) dengan menambahkan dua dimensi tambahan: halusinasi **faktualitas** (*factuality hallucination*) dan halusinasi **kefidelan** (*faithfulness hallucination*). Halusinasi faktualitas terjadi ketika keluaran tidak sesuai dengan fakta dunia nyata, sedangkan halusinasi kefidelan terjadi ketika keluaran menyimpang dari instruksi pengguna, konteks, atau logika internal model. Rawte et al. (2024) bahkan mengajukan dua variasi tambahan, yakni *factual mirage* (jawaban salah terhadap pertanyaan yang benar) dan *silver lining* (jawaban tampak benar terhadap pertanyaan yang salah) (**math2024hallucination**).

Dalam konteks *Retrieval-Augmented Generation* (RAG), batasan pengetahuan internal LLM dapat dikompensasi dengan pengambilan informasi eksternal. Namun, halusinasi tetap dapat muncul ketika sumber hasil *retrieval* mengandung data yang salah, sudah usang, atau tidak relevan. Misalnya, apabila dokumen hasil pencarian mengandung informasi yang keliru, maka model dapat menghasilkan *factual hallucination*. Sebaliknya, jika kueri pengguna ambigu, maka sistem dapat salah menafsirkan konteks dan menghasilkan *faithfulness hallucination* (**math2024hallucination**).

Tabel II.1 Contoh berbagai jenis *hallucination* pada *Large Language Models (LLM)*

Jenis *Hallucination*	**Contoh Pertanyaan / Instruksi**	**Keluaran Model**	**Penjelasan**
Halusinasi Intrinsik	”Laporan keuangan Perusahaan A menunjukkan arus kas operasi sebesar CNY 520 juta. Analisislah kondisi arus kas Perusahaan A.”	”Arus kas bersih dari aktivitas investasi sebesar CNY 380 juta.”	Keluaran model bertentangan dengan informasi numerik pada *prompt*. Nilai yang dihasilkan tidak sesuai dengan angka yang diberikan, sehingga menunjukkan ketidakselarasan konteks internal terhadap data awal.

Bersambung pada halaman berikutnya

Tabel II.1 Contoh jenis - jenis *hallucination* pada *Large Language Models* (lanjutan)

Jenis *Hallucination*	**Contoh Pertanyaan / Instruksi**	**Keluaran Model**	**Penjelasan**
Halusinasi Ekstrinsik	”Laporan keuangan Perusahaan A menunjukkan arus kas operasi sebesar CNY 520 juta. Analisislah kondisi arus kas Perusahaan A.”	”Peningkatan arus kas disebabkan oleh ekspansi ke pasar Asia Tenggara.”	Model menghasilkan penjelasan berbasis informasi eksternal yang tidak tercantum pada dokumen sumber. Secara linguistik dan logis jawaban tampak konsisten, namun tidak didukung konteks faktual pada data input.
Halusinasi Faktualitas	”Sebutkan nama presiden Amerika Serikat saat ini.”	”Presiden Amerika Serikat saat ini adalah William Howard Taft.”	Keluaran model mengandung klaim faktual yang keliru. Nama pejabat publik yang disebutkan tidak merepresentasikan realitas politik aktual, menunjukkan kesalahan substansi meskipun format kalimat tersusun baik.
Halusinasi Ketidakselarasan Tugas	”Terjemahkan: Who was the director of *Avatar*?”	”*Avatar* disutradarai oleh James Cameron.”	Model tidak mengikuti instruksi utama yang diminta, yaitu melakukan penerjemahan. Model justru menjawab isi pertanyaan. Ketidaktaatan terhadap tujuan instruksi menunjukkan kegagalan pemahaman arah tugas.

Bersambung pada halaman berikutnya

Tabel II.1 Contoh jenis - jenis *hallucination* pada *Large Language Models* (lanjutan)

Jenis *Hallucination*	**Contoh Pertanyaan / Instruksi**	**Keluaran Model**	**Penjelasan**
Factual Mirage	”Sir Arthur Conan Doyle adalah penulis seri Sherlock Holmes. Jelaskan penghargaan resmi yang diterimanya.”	”Arthur Conan Doyle menerima penghargaan ’Royal Detective Honor’ dari Kerajaan Inggris.”	Model memproduksi entitas fiktif yang tampak kredibel dan relevan dengan konteks. Padahal, penghargaan tersebut tidak pernah ada. Fenomena ini mencerminkan pembentukan fakta semu yang sulit dibedakan dari informasi nyata.
Silver Lining	”Mengapa Lin Daiyu mencabut pohon willow?”	”Lin Daiyu mencabut pohon willow sebagai simbolisasi kesedihan mendalam atas nasib tragisnya.”	Model menjawab berdasarkan asumsi naratif dari teks yang diberikan, padahal tindakan yang menjadi premis dalam *prompt* tidak pernah terjadi pada karya aslinya. Ketika dipaksa menerima pernyataan yang salah, model memvalidasi konteks fiktif dan mengembangkan narasi lanjutan.

Fenomena halusinasi pada LLM dapat disebabkan oleh tiga faktor utama:

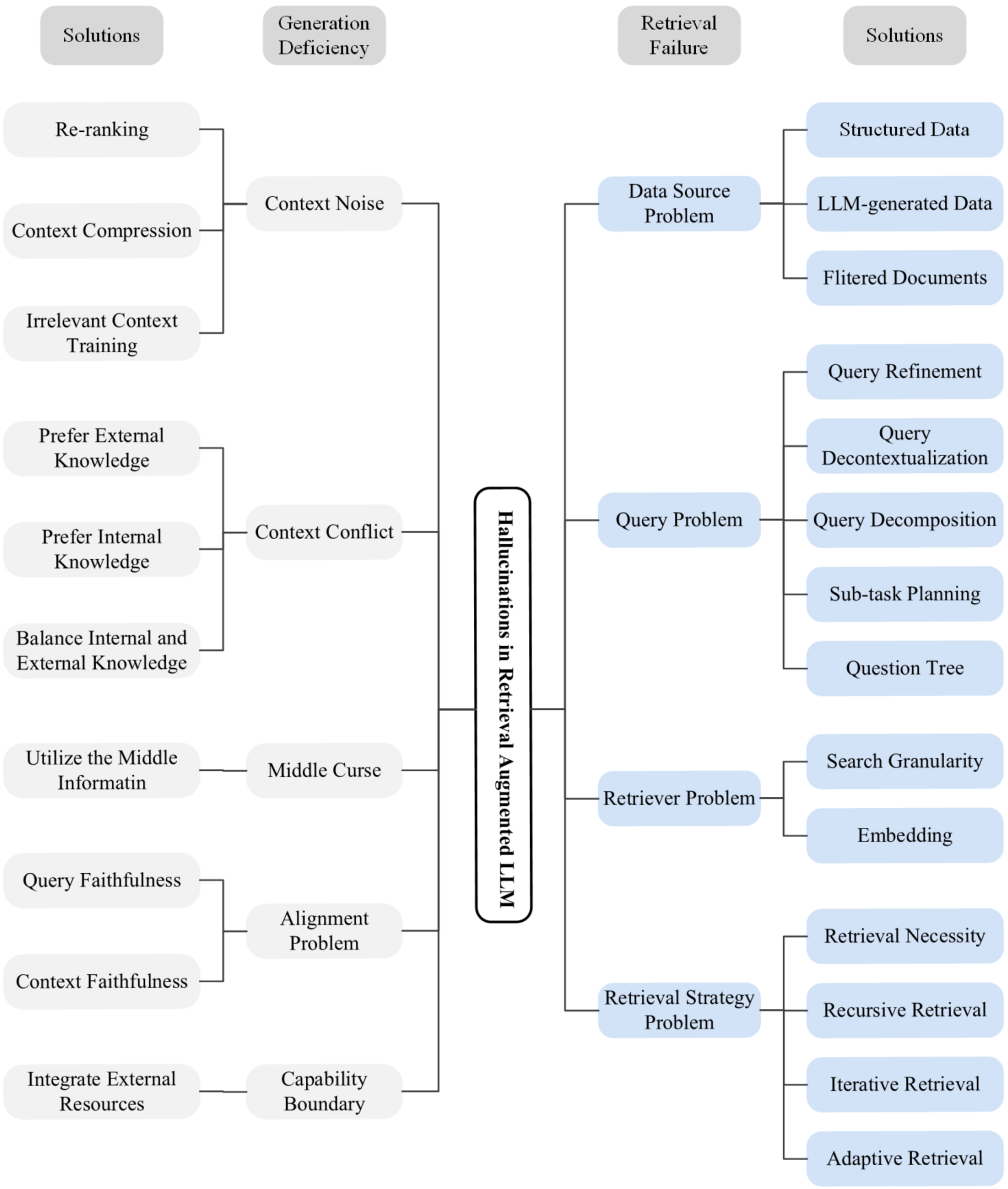
a. keterbatasan pengetahuan internal model akibat cakupan data pelatihan yang terbatas;
b. ketidaksesuaian antara kueri pengguna dan hasil pencarian pada tahap *retrieval*;
c. kesalahan integrasi atau interpretasi konteks eksternal pada tahap *generation*.

Dengan demikian, deteksi dan mitigasi halusinasi menjadi aspek penting dalam pengembangan sistem berbasis LLM, khususnya pada domain yang menuntut keakuratan dan kredibilitas tinggi seperti kesehatan, keuangan, serta pendidikan (**math2024hallucination**).

II.2 Mitigasi Halusinasi pada *Retrieval-Augmented Generation*

Tinjauan oleh Math dkk. (2025) memberikan pemetaan komprehensif mengenai fenomena *hallucination* pada *Retrieval-Augmented Generation* (RAG), mulai dari sumber penyebab di setiap tahap *pipeline* hingga teknik mitigasi yang telah diusulkan dalam literatur (**math2024hallucination**). RAG sendiri dirancang untuk mengurangi ketergantungan LLM pada pengetahuan parametris dengan menambahkan konteks eksternal, tetapi dalam praktiknya sistem ini tetap rentan menghasilkan keluaran yang tidak akurat ketika terjadi kegagalan pada tahap *retrieval* maupun *generation*.

Secara garis besar, *hallucination* pada RAG dikelompokkan ke dalam dua sumber utama: ***retrieval failure*** dan ***generation deficiency***. *Retrieval failure* muncul ketika sistem gagal menemukan atau memilih informasi yang relevan, misalnya karena kueri ambigu, pemeringkatan dokumen yang buruk, atau korpus yang berisi data usang. Sementara itu, *generation deficiency* berkaitan dengan ketidakmampuan model untuk menyelaraskan jawaban dengan konteks yang sudah benar, misalnya akibat panjang konteks, konflik antar sumber, atau desain *prompt* yang tidak tegas (**math2024hallucination**).



Gambar II.2 Penyebab dan solusi halusinasi pada RAG-LLM (**math2024hallucination<empty citation>**).

Bagian ini menguraikan kerangka RAG yang dibahas oleh Math dkk. (2025), teknik mitigasi yang diusulkan pada setiap subtugas, serta ringkasan metrik dan dataset yang digunakan untuk mengevaluasi keberhasilan mitigasi tersebut, sebagaimana dirangkum pada publikasi tersebut.

II.2.1 Kerangka RAG dan Titik Munculnya *Hallucination*

Math dkk. memodelkan sistem RAG sebagai rangkaian subtugas yang tersebar di beberapa tahap utama (**math2024hallucination**):

1. **Pemahaman kueri** (*query understanding*) Sistem menafsirkan pertanyaan

atau instruksi pengguna dan membentuk kueri yang akan digunakan pada modul *retrieval*. Ambiguitas, ketidaklengkapan, atau pemilihan kata yang keliru pada tahap ini dapat berujung pada *retrieval failure*.

2. **Pengambilan dokumen** (*document retrieval*) Berdasarkan kueri, *retriever* memilih dokumen atau potongan teks (*passage*) yang dianggap paling relevan dari korpus. Kinerja yang buruk pada tahap ini menghasilkan konteks yang tidak relevan, tidak lengkap, atau bias.
3. **Seleksi dan pemrosesan konteks** (*context selection and processing*) Sistem memilih subset konteks yang akan dimasukkan ke dalam *prompt*, sering kali dengan melakukan pemotongan, pemeringkatan ulang, atau kompresi. Di sini dapat muncul masalah seperti konteks terlalu panjang, informasi penting terpotong, atau kebisingan yang tidak tersaring.
4. **Integrasi pengetahuan** (*knowledge integration*) LLM menggabungkan pengetahuan parametris internal dengan konteks eksternal yang diambil melalui *retrieval*. Konflik antara pengetahuan internal dan eksternal, atau kegagalan dalam mengutamakan bukti dari konteks, dapat memicu *hallucination*.
5. **Generasi jawaban** (*answer generation*) Pada tahap ini, model menghasilkan teks akhir yang akan diterima pengguna. Desain *prompt*, strategi decoding, dan tidak adanya mekanisme verifikasi dapat membuat model memproduksi pernyataan yang tidak didukung dokumen.

Dengan memetakan subtugas seperti di atas, Math dkk. menekankan bahwa mitigasi halusinasi tidak bisa hanya dilakukan dengan satu teknik pada satu tahap, melainkan memerlukan *pipeline* yang secara eksplisit menangani sumber kesalahan di tahap *retrieval* maupun *generation*.

II.2.2 Mitigasi Halusinasi pada Tahap *Retrieval*

Pada tahap *retrieval*, halusinasi umumnya berasal dari kueri yang tidak representatif serta pemeringkatan dokumen yang tidak tepat. Math dkk. mengelompokkan teknik mitigasi utama sebagai berikut (**math2024hallucination**).

II.2.2.1 Perbaikan Kueri dan *Query Rewriting*

Jika kueri pengguna terlalu pendek, ambigu, atau mengandung istilah yang tidak spesifik, *retriever* dapat mengembalikan dokumen yang tidak relevan. Untuk mengurangi masalah ini, beberapa studi mengusulkan:

- ***Query rewriting*** berbasis LLM, yaitu menulis ulang pertanyaan pengguna menjadi kueri yang lebih terstruktur, eksplisit, dan kaya konteks.
- **Ekspansi kueri** (*query expansion*) dengan menambahkan sinonim, istilah teknis terkait, atau entitas yang relevan.

- ***Decontextualization***, yaitu mengubah pertanyaan yang bergantung pada konteks percakapan menjadi bentuk mandiri sehingga lebih mudah dicocokkan dengan dokumen.

Teknik-teknik ini bertujuan mengurangi *retrieval failure* yang muncul semata-mata karena formulasi kueri yang kurang baik. Dalam konteks tugas akhir ini, modul *query rewriting* dapat ditempatkan sebelum *retriever* untuk memastikan bahwa kueri yang masuk ke tahap berikutnya sudah dioptimalkan.

II.2.2.2 Peningkatan Kualitas *Retriever* dan *Reranking*

Selain memperbaiki kueri, banyak penelitian fokus pada peningkatan kualitas *retriever* itu sendiri. Pendekatan yang dibahas dalam tinjauan tersebut meliputi:

- Penggunaan ***dense retriever*** berbasis *embedding*, yang menangkap kedekatan semantik di luar kecocokan kata kunci.
- ***Hybrid retrieval*** yang menggabungkan metode klasik seperti BM25 dengan *dense retriever* untuk menyeimbangkan presisi dan jangkauan.
- ***Reranking*** dokumen menggunakan model yang lebih kuat (misalnya model *cross-encoder*) untuk menilai kembali relevansi top-k hasil awal.

Reranking menjadi penting ketika *retriever* bekerja pada skala besar: alih-alih mencoba mengoptimalkan seluruh daftar hasil, sistem fokus pada memeringkat ulang sejumlah kecil kandidat teratas. Dengan demikian, konteks yang akhirnya dimasukkan ke *prompt* memiliki probabilitas lebih tinggi untuk benar-benar relevan, yang pada gilirannya mengurangi risiko *hallucination* di tahap *generation*.

II.2.2.3 Seleksi dan Kompresi Konteks

Masalah lain yang disorot adalah *lost in the middle*, di mana LLM cenderung mengabaikan bagian tengah konteks yang panjang. Untuk mengatasi hal ini, beberapa teknik yang dibahas antara lain:

- **Kompresi semantik** (*semantic compression*), misalnya melalui model seperti *LLMLingua*, yang merangkum konteks menjadi bentuk lebih ringkas namun tetap informatif.
- **Seleksi kontekstual** (*selective context selection*), yaitu memilih hanya kalimat atau paragraf yang paling relevan terhadap pertanyaan.
- ***Chunking* dan pengurutan adaptif**, di mana dokumen dipecah menjadi segmen-segmen kecil dan diurutkan ulang berdasarkan kedekatan dengan kueri.

Teknik seleksi dan kompresi ini menurunkan tingkat kebisingan pada konteks, sehingga model tidak terdorong membuat asumsi ketika berhadapan dengan informasi yang tidak relevan atau terlalu banyak.

II.2.3 Mitigasi pada Tahap Integrasi Pengetahuan

Pada tahap integrasi pengetahuan, LLM harus menggabungkan pengetahuan internal dengan bukti eksternal. Math dkk. menyoroti beberapa tantangan utama: konflik antar sumber, konsistensi logika, dan pemilihan bukti yang tepat (**math2024hallucination**).

Untuk mengurangi *hallucination* pada tahap ini, beberapa pendekatan yang dibahas antara lain:

- **Konsolidasi pengetahuan** (*knowledge consolidation*), di mana sistem menyusun representasi terstruktur dari bukti sebelum generasi jawaban akhir.
- **Resolusi konflik** antar dokumen dengan meminta model mengidentifikasi perbedaan dan memilih bukti yang paling kredibel.
- **Mekanisme memori** (*memory mechanisms*) yang menyimpan fakta penting dari konteks sehingga tidak hilang ketika model memproses konteks yang panjang.

Meskipun tugas akhir ini tidak secara langsung mengimplementasikan modul integrasi pengetahuan khusus, pemahaman terhadap tahap ini penting untuk menjelaskan bahwa mitigasi di tingkat *retrieval* dan *generation* perlu didesain agar selaras.

II.2.4 Mitigasi Halusinasi pada Tahap *Generation*

Pada tahap *generation*, Math dkk. mengkaji berbagai teknik yang berfokus pada cara model menghasilkan dan memverifikasi jawaban (**math2024hallucination**). Beberapa kategori utama adalah sebagai berikut.

II.2.4.1 *Prompt Engineering* dan Strategi Penalaran

Berbagai teknik *prompting* digunakan untuk memandu model agar lebih berhati-hati, antara lain:

- ***Chain-of-thought*** dan penalaran bertahap yang mendorong model menjelaskan langkah berpikirnya.
- ***Vigilant prompting*** yang secara eksplisit menginstruksikan model untuk hanya menggunakan informasi dari konteks dan menyatakan "tidak tahu" jika bukti tidak memadai.
- **Teknik lanjutan** seperti *ReAct*, CoVe, CoN, dan CoK yang memadukan penalaran, aksi, verifikasi, dan pengelolaan dokumen untuk menekan *hallucination*.

Teknik-teknik ini tidak mengubah parameter model, tetapi mengatur perilaku generasi melalui desain *prompt* yang cermat.

II.2.4.2 Verifikasi Jawaban dan *Active Detection*

Selain mengarahkan proses generasi, banyak tugas akhir yang menambahkan lapisan verifikasi setelah jawaban dihasilkan. Contoh pendekatan yang dibahas dalam tinjauan tersebut meliputi:

- **Pemeriksaan berbasis konsistensi**, seperti *SelfCheckGPT*, yang membandingkan beberapa sampel generasi untuk menilai stabilitas klaim (**selfcheckgpt2023**).
- **Verifikasi berbasis sumber**, misalnya dengan memformulasikan ulang jawaban menjadi serangkaian pertanyaan dan memeriksanya terhadap konteks *retrieval*.
- **Deteksi segmen halusinatif**, di mana model atau klasifikator tambahan menandai kalimat atau frasa yang tidak didukung bukti, untuk kemudian ditulis ulang atau dihapus.

Pendekatan ini sejalan dengan konsep *active detection* yang akan dimanfaatkan dalam tugas akhir ini sebagai lapisan terakhir untuk menandai dan, bila perlu, memperbaiki keluaran yang berisiko mengandung *hallucination*.

II.3 *Query Rewriting* untuk Mitigasi Halusinasi pada RAG

Salah satu penyebab paling umum dari halusinasi dalam sistem *Retrieval-Augmented Generation* (RAG) adalah kegagalan pada tahap *retrieval*, terutama ketika kueri pengguna bersifat pendek, ambigu, atau tidak memiliki kata kunci yang sesuai dengan struktur dokumen dalam korpus. Untuk mengatasi kelemahan ini, Ma dkk. (2023) mengusulkan pendekatan ***Query Rewriting for Retrieval-Augmented Large Language Models***, yaitu teknik untuk mentransformasikan kueri mentah pengguna menjadi bentuk kueri baru yang lebih jelas, lebih informatif, dan lebih mudah dicocokkan oleh *retriever* (**ma2023queryrewriting**).

Pendekatan ini bekerja dengan memanfaatkan kemampuan *Large Language Models* dalam menafsirkan maksud pengguna secara lebih mendalam, kemudian menghasilkan ulang kueri dalam bentuk yang kaya konteks. Tujuannya bukan sekadar memperpanjang kueri, melainkan:

1. mengekspansi istilah penting yang tersirat,
2. menambahkan kata kunci domain yang relevan,
3. mendisambiguasi istilah atau objek yang memiliki lebih dari satu makna,
4. menghilangkan ketidakjelasan linguistik,
5. menyelaraskan format kueri dengan gaya penulisan korpus rujukan.

Ma dkk. menunjukkan bahwa *retriever* tradisional seperti BM25 atau Dense Passage Retrieval sering kali gagal menemukan dokumen yang benar ketika kueri terlalu pen-

dek atau tidak selaras dengan distribusi bahasa pada korpus. Dengan memperbaiki bentuk kueri melalui proses *rewriting*, sistem RAG dapat meningkatkan **recall** dokumen relevan secara signifikan, sehingga mengurangi risiko halusinasi pada tahap generasi karena model tidak perlu ”mengarang” informasi untuk mengisi kekosongan konteks.

Secara teknis, pendekatan ini membingkai *query rewriting* sebagai fungsi:

$$q' = \text{LLM_rewrite}(q),$$

di mana q adalah kueri awal pengguna, dan q' adalah kueri terstruktur yang dihasilkan LLM. Kueri baru tersebut kemudian digunakan oleh *retriever* untuk mendapatkan konteks:

$$C = \text{Retriever}(q').$$

Contoh Transformasi Kueri

Berikut beberapa contoh hasil transformasi kueri menggunakan *query rewriting*:

- **Kueri awal:** “How do I keep my plants alive while I’m on holiday?”
  Kueri baru: “What are the best methods for ensuring my indoor and outdoor plants remain healthy and watered during a vacation?”
- **Kueri awal:** “Which country had the most athletes in the Olympic parade?”
  Kueri baru: “Which country had the largest number of athletes participating in the most recent Olympic opening ceremony parade?”
- **Kueri awal:** “Why were there concerns about the Seine?”
  Kueri baru: “What specific issues or concerns have arisen regarding the Seine River, such as pollution, flooding, or ecological impacts?”

Berdasarkan eksperimen dalam Ma dkk. (2023), teknik ini memberikan keuntungan berikut:

- **Peningkatan relevansi konteks** melalui kueri yang lebih kaya informasi.
- **Penurunan *retrieval failure***, khususnya pada kueri pendek.
- **Peningkatan *faithfulness*** jawaban karena LLM memperoleh konteks yang lebih akurat.
- **Reduksi halusinasi generatif**, karena model tidak lagi bergantung pada memori parametrik semata.

Dalam konteks ini, *query rewriting* berfungsi sebagai **mekanisme mitigasi pada tahap retrieval**. Dengan mengolah ulang kueri pengguna sebelum proses pencarian dokumen, sistem RAG menjadi lebih stabil dan lebih tahan terhadap halusinasi baik intrinsik maupun ekstrinsik.

II.4 *Reranking* Konteks untuk Mengurangi Kebisingan

Pada sistem *Retrieval-Augmented Generation* (RAG), tahap *retrieval* sering menghasilkan sekumpulan potongan dokumen (*chunk*) yang tidak hanya relevan, tetapi juga mengandung informasi yang berulang, terlalu umum, atau bahkan tidak terkait dengan maksud kueri. Kondisi ini dikenal sebagai *konteks berisik* dan dapat mendorong *Large Language Model* (LLM) untuk melakukan *hallucination*, karena model berusaha menyintesis jawaban dari bukti yang sebagian tidak tepat sasaran.

Ma dkk. mengkaji masalah ini pada skenario kueri yang bersifat *multi-faceted*, yaitu ketika satu kueri pengguna memuat beberapa aspek atau *sub-intent* sekaligus, misalnya dimensi produk, atribut kualitas, dan preferensi pengguna. Dalam kasus seperti ini, *retriever* standar (misalnya *BM25* atau *dense retriever*) cenderung mengembalikan *top-k* dokumen yang hanya menangkap sebagian aspek kueri dan mencampurkannya dengan dokumen yang relevansinya lemah.

Untuk mengurangi kebisingan dan meningkatkan kualitas konteks yang dikonsumsi oleh LLM, Ma dkk. mengusulkan modul *context reranking*. Alih-alih langsung mengirim semua hasil *retrieval* ke model generatif, sistem terlebih dahulu menerapkan tahap seleksi ulang (*reranking*) terhadap *top-k* kandidat. Secara garis besar, prosesnya meliputi:

1. **Pengambilan kandidat awal**
   Retriever menghasilkan *top-k* potongan dokumen berdasarkan kemiripan awal terhadap kueri, misalnya dengan *BM25*, *dense retrieval*, atau *vector database*. Hasil pada tahap ini masih berpotensi berisik dan tidak seimbang antar aspek kueri.
2. **Penghitungan skor relevansi semantik**
   Setiap *chunk* kandidat diberi skor relevansi terhadap representasi kueri yang diperkaya, tidak hanya berdasarkan kecocokan leksikal, tetapi juga kesesuaian semantik terhadap berbagai aspek kueri (**craft2023richresponses**). Hal ini penting untuk menangkap bagian kueri yang bersifat *multi-faceted*.
3. ***Multi-facet scoring* dan penalti kebisingan**
   Modul *reranker* mengombinasikan beberapa komponen skor, antara lain: relevansi inti terhadap kueri, cakupan atribut atau aspek kueri, serta penalti untuk *chunk* yang terlalu umum, redundant, atau tidak spesifik. Dengan cara ini, *chunk* yang hanya mengandung informasi generik atau keluar dari konteks akan ditempatkan di peringkat bawah.
4. **Pemilihan *top-m* konteks terpilih**
   Berdasarkan skor komposit, sistem memilih *top-m chunk* terbaik sebagai kon-

teks akhir yang akan diberikan kepada LLM. Nilai m biasanya lebih kecil daripada k sehingga konteks menjadi lebih ringkas, fokus, dan relevan.

5. **Generasi jawaban berbasis konteks yang telah disaring**
   LLM kemudian menghasilkan jawaban dengan mengandalkan *top-m* konteks terpilih tersebut. Karena kebisingan berkurang dan sudut pandang kueri tercakup dengan lebih seimbang, kecenderungan *hallucination* dapat ditekan.

Pendekatan *context reranking* ini relevan karena berfokus pada deteksi dan mitigasi *hallucination* pada sistem RAG. Pada tahap *retrieval*, modul *reranker* berperan sebagai filter kualitas yang memastikan hanya konteks yang paling relevan dan informatif yang diteruskan ke model generatif. Dengan demikian, *hallucination* yang disebabkan oleh *retrieval failure* berupa pemilihan dokumen yang salah atau berisik dapat dikurangi sebelum memasuki tahap *generation* (**math2024hallucination**; **craft2023richresponses**).

II.5 *Active Detection* untuk Mitigasi *Hallucination*

Varshney dkk. (2023) mengusulkan pendekatan *active detection* sebagai mekanisme deteksi dan mitigasi *hallucination* secara langsung selama proses generasi, bukan setelah teks sepenuhnya dihasilkan. Ide utamanya adalah bahwa *Large Language Model* (LLM) menunjukkan pola tertentu ketika berada pada kondisi berisiko tinggi menghasilkan *hallucination*, terutama ketika model menghasilkan token dengan tingkat kepercayaan rendah. Dengan melakukan pemeriksaan terhadap bagian yang memiliki *low-confidence*, sistem dapat menghentikan proses generasi sementara dan memicu langkah verifikasi atau koreksi sebelum melanjutkan (**varshney2023stitch**).

Pendekatan ini sangat relevan dalam konteks tugas akhir mengenai deteksi dan mitigasi *hallucination* pada sistem *Retrieval-Augmented Generation* (RAG), karena banyak *hallucination* muncul pada bagian tertentu dari keluaran model, terutama ketika konteks eksternal kurang kuat atau ambigu.

II.5.1 Prinsip Dasar *Active Detection*

Metode *active detection* berangkat dari dua pengamatan kunci:

1. **Token berisiko tinggi biasanya memiliki probabilitas rendah**. Model cenderung berada pada kondisi tidak pasti ketika menemukan konteks yang tidak jelas, dan hal ini tercermin melalui distribusi probabilitas token yang melebar. Token yang dipilih pada kondisi ini berpotensi menyebabkan *hallucination*.
2. **Verifikasi lokal dapat mencegah kesalahan global**. Dengan menghentikan generasi pada momen ketidakpastian dan melakukan pemeriksaan tambahan, sistem mampu memperbaiki kesalahan sebelum menjadi bagian dari keluaran

akhir.

II.5.2 Alur Kerja *Active Detection*

Berdasarkan Varshney dkk. (2023), proses *active detection* mencakup tiga langkah utama:

1. **Identifikasi token berisiko rendah kepercayaan** Model memantau setiap langkah generasi dan menghitung nilai probabilitas token yang dihasilkan. Jika nilai tersebut berada di bawah ambang tertentu, sistem menandai posisi tersebut sebagai titik risiko.
2. **Validasi token yang ditandai** Untuk token yang ditandai, sistem menjalankan proses verifikasi, misalnya melalui:
   - *self-consistency check*, yaitu model diminta menghasilkan beberapa alternatif untuk menilai kestabilan prediksi,
   - *entailment check* jika konteks tersedia,
   - pembandingan semantik antara kandidat token dan konteks sumber.
3. **Koreksi sebelum melanjutkan generasi** Jika token berisiko dianggap tidak valid atau tidak didukung oleh konteks, model memilih alternatif token dengan probabilitas yang lebih tinggi atau mengulang prediksi. Dengan cara ini, kesalahan dicegah sebelum berkembang menjadi *hallucination* yang lebih besar.

II.5.3 Kelebihan Metode *Active Detection*

Pendekatan ini menawarkan beberapa manfaat penting:

- **Mitigasi proaktif**. Berbeda dengan metode pascagenerasi, *active detection* mencegah *hallucination* saat proses generasi berlangsung.
- **Tidak memerlukan model verifikator besar**. Seluruh proses berjalan dengan memanfaatkan informasi internal model, terutama distribusi probabilitas token.
- **Dapat dikombinasikan dengan RAG**. Dalam sistem RAG, metode ini dapat melengkapi pendekatan lain seperti *context reranking* atau *query rewriting*, sehingga mitigasi dapat dilakukan baik pada sisi *retrieval* maupun pada saat generasi (**math2024hallucination**).

II.5.4 Keterbatasan

Walaupun efektif, *active detection* memiliki beberapa keterbatasan:

- **Membutuhkan akses probabilitas token**. Pendekatan ini hanya dapat diterapkan pada model yang menyediakan *logits* atau probabilitas token, sehingga tidak berlaku untuk model *closed-source* tertentu.

- **Penentuan ambang yang sensitif**. Ambang probabilitas terlalu rendah gagal mendeteksi kesalahan, sedangkan ambang terlalu tinggi menyebabkan verifikasi berlebihan yang meningkatkan latensi.
- **Tidak selalu menangkap halusinasi semantik kompleks**. Token dengan probabilitas tinggi pun dapat menghasilkan klaim salah jika konteks *retrieval* tidak relevan.

II.6 *Prompt Engineering* untuk Reduksi *Hallucination*

Selain arsitektur model dan kualitas data, perancangan *prompt* juga berperan besar dalam memicu maupun mengurangi *hallucination* pada LLM. Studi survei oleh Sahoo dkk. (2024) menunjukkan bahwa *prompt engineering* tidak hanya berfungsi sebagai antarmuka instruksi, tetapi juga sebagai mekanisme pengendali perilaku model, termasuk dalam mengarahkan model agar lebih berhati-hati terhadap fakta dan sumber informasi (**sahoo2024prompt**).

Secara umum, teknik *prompting* untuk mengurangi *hallucination* dapat dikelompokkan ke dalam beberapa pendekatan: (i) memperkaya konteks dengan informasi yang relevan melalui *Retrieval-Augmented Generation* (RAG), (ii) menggabungkan penalaran dan aksi secara eksplisit seperti pada *ReAct*, (iii) menambahkan tahapan verifikasi terstruktur seperti pada *Chain-of-Verification (CoVe)*, (iv) mengelola dokumen hasil *retrieval* yang berisik seperti pada *Chain-of-Note (CoN)*, dan (v) mengintegrasikan pengetahuan secara bertahap seperti pada *Chain-of-Knowledge (CoK)* (**sahoo2024prompt**). Pendekatan-pendekatan ini memanfaatkan *prompt* bukan hanya sebagai instruksi tunggal, melainkan sebagai rangkaian langkah yang membantu model menelusuri bukti, memeriksa konsistensi, dan menolak menjawab ketika informasi tidak memadai.

Pada RAG, *prompt* dirancang untuk memasukkan potongan dokumen hasil *retrieval* ke dalam instruksi sehingga model terdorong mendasarkan jawabannya pada konteks eksternal, bukan hanya pada memori parametrik. *Prompt* yang menekankan frasa seperti "gunakan hanya informasi dari konteks berikut" atau "kutip sumber yang mendukung jawaban" terbukti meningkatkan *faithfulness* karena model diarahkan untuk menghasilkan jawaban berbasis bukti (**lewis2020retrieval**; **ragtruth2025**).

Pendekatan *ReAct* (*Reasoning and Acting*) memadukan langkah penalaran (*thought*) dan tindakan (*action*) di dalam *prompt*. Alih-alih langsung menghasilkan jawaban akhir, LLM diminta untuk menuliskan jejak penalaran singkat, memutuskan kapan perlu memanggil alat eksternal (misalnya API pencarian), lalu memperbarui rencana berdasarkan informasi baru (**yao2022react**). Pola *prompt* seperti "*Thought: ...*",

"*Action: Search[...]*", dan "*Observation: ...*" membantu model menyadari keterbatasannya dan mengurangi kecenderungan mengarang informasi.

Chain-of-Verification (CoVe) menambahkan tahap verifikasi eksplisit. *Prompt* meminta model untuk: (i) menghasilkan jawaban awal, (ii) menyusun daftar pertanyaan verifikasi, (iii) menjawab pertanyaan tersebut secara independen, dan (iv) merevisi jawaban sesuai hasil verifikasi (**dhuliawala2024cove**). Pendekatan ini meniru metode pemeriksaan ulang yang dilakukan manusia, sehingga efektif dalam mengurangi *hallucination* faktual.

Chain-of-Note (CoN) berfokus pada pengelolaan dokumen hasil *retrieval* yang berisik. Melalui *prompt* yang meminta model menyoroti poin penting, mengabaikan dokumen tidak relevan, serta menandai ketidakpastian, CoN menjadikan model lebih selektif terhadap bukti yang digunakan (**yu2024con**).

Sementara itu, *Chain-of-Knowledge (CoK)* memecah masalah kompleks menjadi langkah penalaran bertahap. Mulai dari persiapan konteks, pengumpulan bukti dari sumber heterogen, hingga integrasi dan koreksi progresif. *Prompt* mengarahkan model untuk tidak langsung melompat ke jawaban akhir, melainkan membangun penalaran yang terstruktur (**li2023cok**). CoK mengurangi *hallucination* yang muncul akibat penalaran terburu-buru dan tidak ter-grounding.

Secara keseluruhan, literatur menegaskan bahwa *prompt* yang terstruktur, eksplisit, dan berlapis lebih mampu mengurangi *hallucination* dibandingkan *prompt* sederhana yang hanya berupa satu instruksi. Dengan demikian, *prompt engineering* merupakan komponen penting dalam rancangan sistem LLM yang andal dan *faithful*.

II.6.1 *Zero-shot* dan *few-shot prompting*

Dua paradigma mendasar dalam *prompting* adalah *zero-shot prompting* dan *few-shot prompting*. Keduanya menjadi fondasi bagi teknik lanjutan seperti *ReAct*, CoVe, CoN, dan CoK.

Zero-shot prompting adalah skenario ketika LLM diminta menyelesaikan suatu tugas hanya berdasarkan instruksi natural tanpa diberikan contoh format sebelumnya. Misalnya instruksi: "Jelaskan perbedaan antara *AHA* dan *BHA* untuk kulit berjerawat." Model mengandalkan kemampuan generalisasi dari *pre-training* dan *instruction tuning*. Pendekatan ini fleksibel, tetapi rentan memicu *hallucination* jika instruksinya ambigu atau tidak menyertakan batasan konteks.

Sebaliknya, *few-shot prompting* memberikan beberapa contoh tugas–jawaban se-

belum model diminta menjawab pertanyaan baru. Contohnya, dua atau tiga contoh penjelasan bahan aktif *skincare* berbasis jurnal dapat disertakan dalam *prompt*. Model kemudian belajar dari pola tersebut melalui mekanisme *in-context learning* (**zhao2023survey**). Ketika contoh yang diberikan *faithful*, teknik ini dapat mengurangi *hallucination* karena model meniru gaya jawaban yang berbasis bukti.

Dalam konteks mitigasi *hallucination*, kedua paradigma ini sering dipadukan dengan instruksi tambahan. Misalnya, *zero-shot* dilengkapi klausa: "jawablah hanya berdasarkan konteks berikut dan nyatakan jika informasi tidak tersedia." Sedangkan *few-shot* dapat menambahkan contoh jawaban yang menunjukkan bagaimana model harus menolak menjawab ketika bukti tidak memadai. Dengan demikian, baik *zero-shot* maupun *few-shot prompting* tidak hanya mengarahkan model memahami tugas, tetapi juga mengelola ketidakpastian dan mencegah *hallucination*.

II.7 Kerangka Kerja Evaluasi Sistem *Retrieval-Augmented Generation (RAG)*

Pengembangan sistem *Retrieval-Augmented Generation* (RAG) memerlukan metode *tuning* yang tepat karena kinerjanya dipengaruhi oleh berbagai komponen seperti model *retriever*, model bahasa (*language model*), rancangan *prompt*, serta parameter konfigurasi lainnya. Oleh karena itu, evaluasi sistem RAG yang terotomasi menjadi hal yang penting dalam memastikan keluaran model yang akurat, relevan, dan berlandaskan bukti (**es2024ragas**).

Berbagai metode dan perkakas evaluasi telah diusulkan oleh beberapa tugas akhir sebelumnya untuk memfasilitasi proses penilaian sistem RAG, misalnya oleh Y. Gao dkk. (2023) dan Es dkk. (2024). Namun demikian, banyak strategi evaluasi yang masih bergantung pada probabilitas token dari model bahasa. Pendekatan tersebut menjadi tidak praktis untuk diterapkan pada model *closed-source* seperti GPT-4, karena model tersebut tidak memberikan akses langsung terhadap probabilitas internalnya. Alternatif lain berupa penilaian berbasis sistem tanya-jawab dengan tipe jawaban singkat juga dinilai tidak representatif terhadap cara kerja sistem RAG secara keseluruhan (**gao2023rag-eval**).

Untuk mengatasi keterbatasan tersebut, dikembangkanlah kerangka kerja bernama **RAGAS** (*Retrieval-Augmented Generation Assessment System*). RAGAS merupakan kerangka kerja evaluasi otomatis yang dirancang untuk menilai kinerja sistem RAG dengan cara memanfaatkan kemampuan LLM dalam melakukan penilaian berbasis *prompt*. Kerangka ini dikembangkan oleh tim peneliti dari Cardiff University dengan tujuan untuk mengukur kualitas sistem RAG dari tiga aspek utama, yaitu *faithfulness*, *answer relevance*, dan *context relevance* (**es2024ragas**).

II.7.1 Aspek Penilaian dalam RAGAS

Aspek *faithfulness* mengukur sejauh mana jawaban yang dihasilkan oleh sistem RAG didasarkan pada konteks yang diberikan. Dalam praktiknya, LLM digunakan untuk mengekstraksi satu atau lebih pernyataan dari setiap kalimat pada jawaban sistem, kemudian memverifikasi apakah pernyataan tersebut terdapat dalam konteks hasil *retrieval*. Nilai *faithfulness* dihitung dengan rumus berikut:

$$F = \frac{|V|}{|S|} \tag{II.1}$$

dengan $|V|$ menyatakan jumlah pernyataan yang didukung oleh konteks (berdasarkan hasil evaluasi LLM), sedangkan $|S|$ adalah jumlah total pernyataan yang dihasilkan sistem RAG.

Aspek kedua, yaitu *answer relevance*, menilai sejauh mana jawaban yang dihasilkan mampu menjawab pertanyaan secara langsung dan tepat. LLM menghasilkan beberapa variasi pertanyaan baru yang diturunkan dari jawaban sistem, kemudian tingkat kesamaan semantik antara pertanyaan-pertanyaan tersebut dan pertanyaan asli dihitung menggunakan ukuran *cosine similarity*. Nilai relevansi jawaban dihitung dengan rumus:

$$AR = \frac{1}{n} \sum_{i=1}^{n} \text{sim}(q, q_i) \tag{II.2}$$

dengan q merupakan pertanyaan asli, q_i adalah pertanyaan yang dihasilkan oleh LLM, dan n menyatakan jumlah pertanyaan yang dihasilkan.

Aspek ketiga adalah *context relevance*, yang berfokus pada relevansi antara konteks hasil *retrieval* dan kebutuhan jawaban. LLM mengekstraksi kalimat-kalimat dari konteks yang relevan untuk menjawab pertanyaan. Skor *context relevance* dihitung melalui rasio antara jumlah kalimat relevan dengan jumlah total kalimat dalam konteks, seperti ditunjukkan pada persamaan berikut:

$$CR = \frac{\text{Jumlah kalimat relevan}}{\text{Jumlah total kalimat dalam konteks}} \tag{II.3}$$

II.7.2 Metrik Turunan dalam RAGAS

a. Context Precision

Metode ini mengukur proporsi segmen (*chunk*) konteks yang relevan terhadap pertanyaan. Nilainya dihitung sebagai rata-rata nilai presisi untuk setiap segmen yang dievaluasi:

$$\text{Context Precision@K} = \frac{1}{K}\sum_{k=1}^{K}(\text{Precision@k} \times v_k) \tag{II.4}$$

dengan K merupakan jumlah total segmen yang diuji dan $v_k \in \{0, 1\}$ adalah indikator relevansi pada segmen ke-k.

Presisi setiap segmen dihitung menggunakan formula berikut:

$$\text{Precision@k} = \frac{\text{True Positives@k}}{\text{True Positives@k} + \text{False Positives@k}} \tag{II.5}$$

b. Context Recall

Metrik ini mengukur seberapa banyak dokumen relevan yang berhasil ditemukan oleh sistem RAG. LLM menghasilkan sejumlah klaim berdasarkan jawaban atau referensi *ground truth*, lalu menghitung proporsi klaim yang dapat dikaitkan dengan konteks yang diambil oleh sistem:

$$\text{Context Recall} = \frac{\text{Jumlah klaim yang sesuai dengan konteks}}{\text{Jumlah total klaim}} \tag{II.6}$$

Pendekatan evaluasi berbasis RAGAS memberikan beberapa keunggulan penting. Pertama, kerangka ini mampu melakukan penilaian yang objektif karena seluruh proses evaluasi dilakukan secara otomatis melalui LLM, sehingga mengurangi bias subjektif dari evaluator manusia. Kedua, RAGAS bersifat adaptif dan dapat diterapkan pada berbagai konfigurasi sistem RAG tanpa memerlukan akses terhadap probabilitas token internal. Ketiga, RAGAS memberikan evaluasi yang komprehensif dengan mempertimbangkan baik aspek *retrieval* maupun *generation*, sehingga menghasilkan pemahaman yang lebih menyeluruh terhadap kinerja sistem.

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

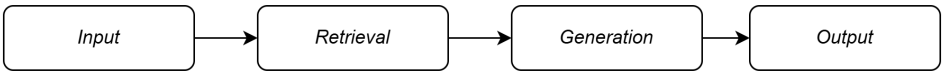
Bab ini menyajikan kondisi faktual terkini dengan kerangka *CRISP-DM* pada dua tahap awal, yaitu *business understanding* dan *data understanding*. Pendekatan ini membantu menegaskan ruang masalah, pemangku kepentingan, risiko, serta karakteristik data sebelum perancangan solusi pada bab berikutnya (**wirth2000crispdm**; **studer2021crispmlq**).

3.1.1 Pemahaman Bisnis (*Business Understanding*)

Model bahasa berskala besar (*Large Language Models*, LLM) telah banyak digunakan untuk tugas penjawaban pertanyaan, dialog, maupun peringkasan. Namun, fenomena *hallucination*, yaitu keluaran yang terdengar meyakinkan tetapi tidak faktual atau tidak didukung bukti, masih banyak dilaporkan pada berbagai model mutakhir (**zhao2023survey**; **math2024hallucination**). Dampaknya signifikan pada domain sensitif seperti kesehatan, hukum, dan keuangan, karena kesalahan faktual berpotensi menurunkan keandalan layanan, meningkatkan risiko operasional, serta menimbulkan konsekuensi etis dan kepatuhan.

Secara umum, sistem *retrieval augmented generation* (RAG) berupaya meminimalkan halusinasi dengan menambahkan tahap pengambilan konteks eksternal yang relevan. Meskipun demikian, hasil studi menunjukkan bahwa masalah tetap muncul akibat keterbatasan relevansi bukti, panjang konteks yang melebihi kapasitas model, serta kegagalan *retriever* dalam memilih dokumen pendukung yang paling tepat (**shuster2021retrieval**). Kondisi tersebut menyebabkan keluaran model tetap berpotensi tidak *faithful* terhadap sumber rujukan.

Struktur umum alur kerja sistem RAG saat ini ditunjukkan pada Gambar III.1. Sistem tersebut terdiri atas tahapan input, *retrieval*, *generation*, dan *output* tanpa adanya komponen *correction* yang secara eksplisit melakukan deteksi maupun mitigasi terhadap *hallucination*.



Gambar III.1 Alur sistem *retrieval augmented generation* (RAG) saat ini

3.1.2 Data Understanding

Tahap pemahaman data dilakukan untuk mengidentifikasi, menelaah, dan membandingkan himpunan data yang relevan sebagai dasar pengujian sistem berbasis *retrieval augmented generation*. Fokus utama pada fase ini adalah memastikan ketersediaan data yang memiliki struktur pertanyaan yang jelas, dukungan konteks tekstual yang terikat langsung pada pertanyaan, serta tingkat kredibilitas sumber informasi yang memadai untuk dijadikan acuan evaluasi.

Sebagai langkah awal, dilakukan penelusuran terhadap dataset yang digunakan secara luas dalam penelitian model bahasa pada beberapa domain yang menuntut tingkat akurasi tinggi, yaitu hukum, kesehatan, dan keuangan. Ketiga domain dipilih karena masing-masing memiliki pola penggunaan *retrieval augmented generation* yang memerlukan informasi referensial terverifikasi. Oleh sebab itu, dataset kandidat harus memuat pertanyaan terstruktur, konteks yang relevan, serta jawaban yang dapat dibandingkan secara objektif terhadap sumber informasi tertulis.

Berikut hasil pemetaan dan perbandingan dataset kandidat dari ketiga domain.

Tabel III.1 Perbandingan Dataset Kandidat

Nama Dataset	**Domain**	**Karakteristik Utama**	**Jumlah Data**
CaseHOLD	Hukum	Pertanyaan mengenai preseden hukum berbasis putusan pengadilan federal Amerika untuk menguji kemampuan model memahami penalaran legal (**casehold**).	$\pm$ 53,000 contoh

Bersambung ke halaman berikutnya

Tabel III.1 Perbandingan Dataset Kandidat (lanjutan)

Nama Dataset	**Domain**	**Karakteristik Utama**	**Jumlah Data**
LegalBench	Hukum	Benchmark komprehensif berisi ratusan subtask untuk evaluasi penalaran hukum, interpretasi pasal, serta ekstraksi norma dan entitas legal (**legalbench**).	Variatif per subtask
COLIEE	Hukum	Dataset berbasis kompetisi internasional untuk ekstraksi informasi dan pembuktian hubungan kasus hukum, mencakup relevansi dokumen dan ranah *legal entailment* (**coliee**).	± 4,000–10,000 contoh
PubMedQA	Kesehatan	Pertanyaan berbasis abstrak ilmiah dalam basis PubMed dengan struktur tanya jawab faktual serta anotasi oleh pakar domain medis (**pubmedqa**).	± 1,000 entri
MedMCQA	Kesehatan	Soal pilihan ganda pendidikan kedokteran mencakup patofisiologi, diagnosis, farmakologi, dan etika klinis yang dikurasi berdasarkan silabus pendidikan medis (**medmcqa**).	± 194,000 soal
MMLU Medical Subset	Kesehatan	Subset khusus medis pada benchmark MMLU yang digunakan untuk evaluasi penguasaan konsep medis, teori klinis, anatomi, serta farmakologi (**mmlu**).	± 80–1,000 pertanyaan per subkategori

Bersambung ke halaman berikutnya

Tabel III.1 Perbandingan Dataset Kandidat (lanjutan)

Nama Dataset	**Domain**	**Karakteristik Utama**	**Jumlah Data**
Financial PhraseBank	Keuangan	Himpunan kalimat terkait laporan keuangan perusahaan publik yang dilengkapi anotasi dan kategorisasi linguistik untuk interpretasi sentimen dan analisis ekonomi (**phrasebank**).	$\pm$ 4,800 kalimat
Banking77	Keuangan	Dataset pertanyaan layanan perbankan yang digunakan secara luas untuk pemetaan intensi pengguna serta evaluasi pemrosesan bahasa dalam layanan finansial digital (**banking77**).	$\pm$ 13,000 contoh

Hasil perbandingan menunjukkan bahwa dataset pada domain hukum dan keuangan memiliki struktur data yang lebih heterogen serta melibatkan beragam bentuk pertanyaan dengan kompleksitas konteks yang tinggi. Pada dataset hukum, pertanyaan sering kali berkaitan dengan preseden yudisial, norma undang-undang, dan proses interpretasi regulasi yang memerlukan pemetaan dokumen lintas kasus. Struktur pertanyaan dan jawaban pada dataset kategori tersebut tidak selalu memiliki keterikatan langsung dengan satu konteks tunggal, sehingga dokumentasi sumber yang relevan bersifat berlapis.

Dataset pada domain keuangan memiliki karakteristik serupa. Sebagian besar himpunan data berorientasi pada interpretasi laporan, sentimen pasar, atau klasifikasi intensi pengguna dalam konteks finansial. Hubungan antara pertanyaan, sumber informasi, dan jawaban tidak selalu bersifat langsung, sehingga validasi faktualitas jawaban terhadap dokumen sumber memerlukan proses pembuktian yang lebih kompleks.

Berbeda dengan kedua domain tersebut, dataset pada ranah kesehatan menawarkan struktur data yang lebih terfokus. Konteks pertanyaan pada dataset kategori ini merujuk pada bukti ilmiah berbasis publikasi dalam jurnal kedokteran, dengan abstrak

riset medis sebagai acuan utama. Salah satu dataset yang paling menonjol dibandingkan kandidat lain adalah PubMedQA.

PubMedQA memiliki karakteristik yang mendukung penerapannya pada penelitian ini. Pertama, setiap entri berisi pertanyaan yang terikat langsung dengan satu abstrak ilmiah dalam basis data PubMed. Hal ini memberikan keterhubungan yang jelas antara pertanyaan, konteks, dan jawaban, sehingga proses pembandingan kesesuaian jawaban terhadap konteks dapat dilakukan secara terukur.

Kedua, PubMedQA memiliki ukuran data yang proporsional untuk kajian akademik. Dengan total sekitar seribu entri, dataset ini representatif untuk menghasilkan analisis berbasis data tanpa memberikan beban komputasi berlebihan.

Ketiga, pertanyaan dalam PubMedQA bersifat faktual dengan format jawaban yang terstandarisasi *(yes, no, dan maybe)*. Struktur tersebut mendukung pelaksanaan evaluasi berbasis kesesuaian terhadap konteks karena derajat kebenaran jawaban dapat ditelusuri secara langsung.

Selain itu, PubMedQA banyak digunakan dalam penelitian *question answering* dan studi *retrieval augmented generation* pada domain medis. Penggunaannya secara luas dalam publikasi ilmiah menjadikan dataset ini memiliki landasan metodologis yang kuat serta mempermudah pembandingan hasil penelitian dengan studi sebelumnya.

Berdasarkan keseluruhan hasil analisis pada tahap pemahaman data, PubMedQA dipandang sebagai himpunan data yang paling sesuai untuk digunakan dalam tugas akhir ini. Struktur pertanyaan yang jelas, keterikatan langsung terhadap konteks ilmiah, ukuran dataset yang proporsional, serta format jawaban yang terstandarisasi menjadikannya kandidat utama dalam proses pengujian.

III.2 Analisis Kebutuhan

Pada tahap ini dilakukan perumusan kebutuhan sistem yang menjadi dasar perancangan arsitektur dan eksperimen pada Tugas Akhir. Analisis kebutuhan bertujuan memastikan bahwa sistem yang dikembangkan mampu mendukung proses mitigasi halusinasi pada *Retrieval-Augmented Generation* (RAG) secara terstruktur dan terukur. Kebutuhan dirumuskan berdasarkan permasalahan yang telah diidentifikasi pada analisis kondisi saat ini, mencakup kegagalan *retrieval*, gangguan *context noise*, dan kecenderungan model menghasilkan keluaran yang tidak *faithful*. Oleh karena itu, kebutuhan sistem dibagi menjadi dua kategori utama, yaitu kebutuhan fungsional yang menggambarkan kapabilitas inti yang harus dimiliki sistem, ser-

ta kebutuhan nonfungsional yang mendefinisikan karakteristik mutu dan batasan operasional sistem. Kedua kelompok kebutuhan ini disajikan pada Tabel III.2 dan Tabel III.3.

Tabel III.2 Kebutuhan Fungsional Sistem RAG-LLM untuk Mitigasi Halusinasi

Kode	**Kebutuhan Fungsional**
F01	Sistem dapat menerima input berupa pertanyaan pengguna dan menghasilkan *query* awal untuk proses *retrieval*.
F02	Sistem dapat melakukan *query rewriting* untuk memperbaiki kualitas *query* sebelum proses pengambilan dokumen.
F03	Sistem dapat melakukan *retrieval* dokumen eksternal berdasarkan *query* yang telah diperbaiki.
F04	Sistem dapat melakukan *context reranking* untuk memilih dokumen yang paling relevan dan mengurangi *context noise*.
F05	Sistem dapat menghasilkan jawaban menggunakan model RAG-LLM berdasarkan konteks yang telah difilter.
F06	Sistem dapat melakukan *active hallucination detection* untuk mendeteksi bagian keluaran yang berpotensi mengandung halusinasi.
F07	Sistem dapat menandai atau memberi label pada segmen jawaban yang terdeteksi sebagai halusinatif.
F08	Sistem dapat menghasilkan laporan evaluasi yang mencakup *accuracy*, *context relevancy*, *context recall*, *faithfulness*, *F1 Score*, serta *hallucination rate*.
F09	Sistem dapat membandingkan keluaran dari beberapa konfigurasi eksperimen, yaitu *baseline*, *query rewriting*, *context reranking*, *active hallucination detection*, dan kombinasi menyeluruh.

Tabel III.3 Kebutuhan Nonfungsional Sistem RAG-LLM untuk Mitigasi Halusinasi

Kode	Kategori	Kebutuhan Nonfungsional
NF01	*Reliability*	Sistem harus menghasilkan proses mitigasi yang konsisten dan stabil pada berbagai set pengujian.
NF02	*Interpretability*	Sistem harus memberikan keluaran yang dapat ditelusuri melalui bukti serta menampilkan bagian halusinatif secara jelas.
NF03	*Reproducibility*	Seluruh eksperimen dapat direproduksi melalui penguncian parameter seperti *temperature*, *top-k*, dan *max tokens*.
NF04	*Portability*	Sistem kompatibel dengan berbagai model LLM dan *pipeline* RAG tanpa ketergantungan vendor tertentu.
NF05	*Ethics and Privacy*	Sistem tidak memproses data pribadi serta mengikuti prinsip *responsible AI*.

III.3 Analisis Pemilihan Solusi

III.3.1 Alternatif Solusi

Berdasarkan kajian literatur dan identifikasi kebutuhan pada sistem *Retrieval-Augmented Generation (RAG)*, terdapat tiga alternatif solusi utama yang dapat diterapkan untuk mengurangi *hallucination*. Setiap solusi mewakili pendekatan berbeda, baik pada tahap *retrieval*, *generation*, maupun *post-generation*. Rincian ketiga alternatif tersebut dijelaskan sebagai berikut.

III.3.1.1 Alternatif Solusi I: Penguatan Tahap *Retrieval* (*Query Rewriting*, *Query Expansion*, dan *Embedding*)

Alternatif solusi pertama berfokus pada penguatan tahap *retrieval*, yaitu tahap yang menentukan kualitas konteks yang akan digunakan oleh model dalam proses generasi. Sejumlah studi menunjukkan bahwa sebagian besar *hallucination* pada sistem RAG berasal dari kegagalan menemukan bukti yang benar, relevan, dan lengkap. Ketidaktepatan *retrieval* menyebabkan model bekerja dengan konteks yang tidak memadai, sehingga meningkatkan peluang model mengisi kekosongan informasi dengan pengetahuan internalnya sendiri. Oleh karena itu, penguatan pada tahap *retrieval* merupakan langkah fundamental dalam menurunkan risiko *hallucination*.

Komponen pertama adalah *query rewriting*, yaitu proses memperbaiki struktur per-

tanyaan pengguna sebelum proses pencarian dokumen dilakukan. Kueri asli sering kali bersifat ambigu, tidak lengkap, atau tidak mencerminkan kebutuhan informasi secara tepat. Melalui *query rewriting*, pertanyaan diubah menjadi representasi yang lebih terstruktur, eksplisit, dan selaras dengan format pengetahuan dalam basis data atau korpus dokumen. Studi mutakhir menunjukkan bahwa teknik ini dapat meningkatkan presisi dokumen yang ditemukan, mengurangi *retrieval error*, serta mencegah masuknya konteks yang tidak relevan yang dapat memicu *hallucination*. Dengan kata lain, *query rewriting* menjadi gerbang awal untuk memastikan proses pencarian bukti berjalan dengan benar.

Komponen kedua adalah *query expansion*, yaitu proses memperluas cakupan kueri dengan menambahkan istilah, sinonim, atau frasa terkait yang relevan dengan maksud pertanyaan. Teknik ini bermanfaat ketika pengguna memberikan kueri yang terlalu sempit atau ketika istilah teknis tertentu memiliki banyak referensi semantik di dalam dokumen. Melalui ekspansi kueri, sistem dapat menjangkau lebih banyak kandidat dokumen yang mungkin mengandung jawaban, sehingga mengurangi risiko kehilangan bukti penting. Pendekatan ini sangat efektif pada domain dengan keragaman istilah tinggi atau pada skenario di mana pengguna tidak mengetahui terminologi yang digunakan dalam basis pengetahuan.

Komponen ketiga adalah penguatan *retrieval* berbasis *embedding*, yang bertujuan memetakan kueri dan dokumen ke dalam ruang vektor sehingga kesamaan semantik dapat diukur secara lebih presisi. Berbeda dengan pencarian berbasis kata kunci, metode *embedding* mampu memahami kesamaan makna meskipun tidak menggunakan kata yang sama. Teknik ini memberikan dasar yang kuat dalam memilih dokumen yang relevan, terutama pada korpus besar yang mengandung variasi bahasa atau sinonim. Dengan penggunaan *embedding* yang tepat, sistem dapat menemukan konteks dengan akurasi semantik yang lebih tinggi, sehingga mengurangi potensi kesalahan akibat dokumen yang tidak berhubungan.

Ketiga komponen ini bekerja saling melengkapi. *Query rewriting* memastikan kueri memiliki struktur yang tepat, *query expansion* memperluas cakupan pencarian agar tidak kehilangan kandidat penting, dan *embedding-based retrieval* memastikan bahwa pemilihan dokumen mempertimbangkan kesamaan semantik secara lebih mendalam. Dengan demikian, risiko *hallucination* dapat ditekan secara signifikan sebelum proses generasi berlangsung.

III.3.1.2 Alternatif Solusi II: Penguatan Tahap *Generation* melalui *Middle Information*, *Context Reranking*, dan *Balance Internal and External Knowledge*

Pendekatan alternatif kedua menitikberatkan pada penguatan tahap *generation*, yaitu tahap ketika model membentuk jawaban akhir berdasarkan konteks yang tersedia. Meskipun informasi yang diretrieval sudah benar dan relevan, sejumlah studi menunjukkan bahwa kualitas generasi tetap dapat menurun apabila model gagal menggunakan seluruh bagian konteks secara efektif. Masalah ini terutama muncul pada skenario dengan konteks panjang, seperti pertanyaan multi-dokumen atau berlapis.

Salah satu penyebab utamanya adalah fenomena middle curse, yaitu kondisi ketika model kesulitan memanfaatkan informasi yang berada di bagian tengah konteks. Mekanisme self-attention pada arsitektur Transformer mengalami penurunan atensi terhadap token yang posisinya tidak berada di awal atau akhir, diperparah oleh keterbatasan resolusi positional encoding dan kapasitas memori ketika memproses konteks panjang. Akibatnya, meskipun informasi penting berada di bagian tengah, model sering melewatkannya sehingga akurasi generasi menurun secara signifikan. Fenomena ini tidak hanya terjadi pada tugas multi-dokumen, tetapi juga pada tugas abstraksi, penulisan jawaban panjang, dan peringkat paragraf.

Untuk mengatasi hal tersebut, teknik pemanfaatan informasi tengah (utilize the middle information) digunakan untuk menonjolkan kembali bagian konteks yang berada di tengah agar tetap terlihat oleh model selama proses generasi. Pendekatan ini dapat berupa penyesuaian bobot atensi, pengulangan segmen penting, atau penyusunan ulang konteks sehingga informasi kritis tidak tenggelam di posisi yang kurang strategis. Dengan menjamin bahwa informasi penting di bagian tengah tetap terakses, risiko kesalahan generasi yang berasal dari hilangnya bukti dapat dikurangi.

Selanjutnya, penguatan relevansi melalui *context reranking* juga diperlukan pada tahap generasi. Meskipun reranking biasanya diterapkan pada tahap *retrieval*, menata ulang konteks berdasarkan relevansi sebelum generasi membantu model memprioritaskan potongan informasi yang lebih penting. Hal ini sangat berguna ketika konteks berasal dari banyak dokumen atau mengandung bukti yang nilai signifikansinya berbeda. Dengan memastikan bahwa bukti paling relevan ditempatkan pada posisi yang mudah diproses model, tingkat kesalahan faktual selama generasi dapat ditekan.

Terakhir, pendekatan *balancing internal and external knowledge* diperlukan untuk mengurangi kecenderungan model mengandalkan pengetahuan internalnya secara

berlebihan. Pada beberapa kasus, model lebih percaya pada memorinya sendiri meskipun sudah diberikan bukti eksternal yang benar. Sebaliknya, ada kasus lain di mana model terlalu terpaku pada konteks meskipun informasi tersebut tidak lengkap. Mekanisme keseimbangan ini bertujuan menuntun model agar membandingkan dan mengonfirmasi jawaban dengan kedua sumber pengetahuan secara seimbang, sehingga hasil akhir lebih konsisten dan faktual.

Ketiga teknik ini saling melengkapi dalam memperkuat struktur pengambilan keputusan model selama tahap generasi. Dengan memastikan bahwa informasi tengah dapat dimanfaatkan, konteks relevan diprioritaskan, dan pengetahuan internal serta eksternal digunakan secara seimbang, proses generasi menjadi lebih stabil dan berbasis bukti. Hal ini berpotensi signifikan dalam menurunkan *hallucination* yang muncul akibat miskonsepsi, hilangnya konteks penting, atau penggunaan bukti yang keliru.

III.3.1.3 Alternatif Solusi III: *Multiple-Phase Mitigation* (*Query Rewriting*, *Context Reranking*, dan *Active Hallucination Detection*)

Alternatif ketiga berfokus pada pendekatan mitigasi berlapis (*multiple-phase mitigation*) yang bekerja pada beberapa tahapan dalam alur RAG, mulai dari tahap *retrieval*, pemilihan konteks, hingga validasi keluaran akhir. Pendekatan ini dirancang untuk mengatasi sumber utama *hallucination*, yaitu kesalahan yang berasal dari kurang jelasnya input pengguna serta kesalahan yang muncul selama proses generasi akibat ketidaksesuaian antara jawaban dan konteks. Dengan menerapkan strategi mitigasi di setiap fase, sistem memperoleh perlindungan yang lebih kuat dan menyeluruh.

Komponen pertama adalah *query rewriting*, yaitu proses memperjelas maksud pertanyaan pengguna sebelum dokumen dicari. Tanpa perbaikan kueri, banyak sistem RAG gagal mengambil konteks yang tepat karena kueri asli pengguna sering ambigu, kurang spesifik, atau tidak mencerminkan kebutuhan informasi yang sebenarnya. Studi terkini menunjukkan bahwa kueri yang telah ditulis ulang dapat meningkatkan kualitas *retrieval* secara signifikan, memperbaiki presisi dokumen yang ditemukan, serta mengurangi risiko terjadinya *hallucination* yang berasal dari konteks yang kurang relevan. Dengan demikian, *query rewriting* berfungsi sebagai gerbang awal yang memastikan sistem memulai proses dengan representasi pertanyaan yang paling akurat (**wang2024maferw**).

Komponen kedua adalah *context reranking*, yaitu penataan ulang dokumen yang berhasil diretrieval berdasarkan tingkat relevansinya terhadap pertanyaan. Meski-

pun sistem berhasil menemukan sejumlah bukti yang benar, tidak semuanya memiliki nilai yang sama. Perbedaan kualitas antardokumen dapat menyebabkan *context conflict* atau *context noise*, di mana bukti yang kurang relevan mengaburkan bukti utama yang diperlukan model. Teknik *reranking* membantu memprioritaskan dokumen dengan dukungan semantik paling kuat dan menempatkannya pada posisi yang lebih mudah diproses model selama tahap generasi. Strategi ini secara langsung mengurangi kemungkinan model menggunakan bukti yang lemah atau bertentangan, khususnya pada skenario multi-dokumen.

Komponen ketiga adalah *active hallucination detection*, yaitu mekanisme untuk memeriksa keluaran model secara langsung ketika jawaban dihasilkan. Pendekatan ini biasanya memanfaatkan indikator ketidaksesuaian antara jawaban dengan konteks, tingkat keyakinan model, atau verifikasi ulang fakta melalui subkueri. Tidak seperti pemeriksaan pasif, deteksi aktif mengidentifikasi bagian jawaban yang berisiko halusinatif secara dinamis, kemudian menandai atau meminta model mengoreksi bagian tersebut sebelum hasil akhir diberikan kepada pengguna.

Ketiga teknik ini membentuk alur mitigasi bertahap yang saling melengkapi. *Query rewriting* mencegah kesalahan sejak tahap pencarian bukti, *context reranking* memastikan hanya bukti terbaik yang menjadi dasar generasi, dan *active hallucination detection* menjaga integritas keluaran akhir. Dengan perlindungan berlapis, risiko kesalahan dapat ditekan dari hulu hingga hilir, menghasilkan sistem RAG yang lebih akurat, lebih dapat dipercaya, dan lebih tahan terhadap variasi pertanyaan serta kompleksitas konteks.

III.3.2 Analisis Penentuan Solusi

Setelah merumuskan beberapa alternatif solusi mitigasi *hallucination* pada sistem *Retrieval-Augmented Generation (RAG)*, langkah berikutnya adalah menentukan solusi yang paling tepat untuk diterapkan dalam tugas akhir ini. Penentuan solusi dilakukan menggunakan pendekatan *weighted scoring*, yaitu metode evaluasi alternatif berdasarkan sejumlah kriteria yang diberi bobot sesuai tingkat kepentingannya. Setiap alternatif diberi skor menggunakan skala *Likert* satu sampai sepuluh untuk masing-masing kriteria, kemudian dihitung skor total tertimbangnya.

Empat kriteria utama yang digunakan adalah sebagai berikut.

Efektivitas menilai sejauh mana suatu alternatif mampu mengurangi *hallucination* pada sistem RAG–LLM. Kriteria ini menjadi prioritas utama karena *hallucination* berdampak langsung pada keakuratan informasi dan kepercayaan pengguna. Tugas

Akhir terdahulu menunjukkan bahwa kesalahan faktual, termasuk misinformasi dalam domain sensitif seperti kesehatan, hukum, dan keuangan, dapat menimbulkan konsekuensi serius bagi individu maupun institusi (**pang2024medical**). Oleh karena itu, alternatif dengan skor efektivitas tinggi diharapkan mampu menurunkan *intrinsic hallucination* dan *factual hallucination* secara signifikan.

Feasibility mengukur sejauh mana alternatif solusi realistis untuk diimplementasikan dalam konteks Tugas Akhir ini, baik dari segi kompleksitas teknis, ketergantungan terhadap infrastruktur, maupun ketersediaan alat dan *library*. Solusi dengan feasibility tinggi bersifat masuk akal untuk diwujudkan pada lingkungan eksperimen dengan sumber daya terbatas, tanpa mengorbankan kualitas mitigasi *hallucination*.

Biaya komputasi mengevaluasi kebutuhan sumber daya yang diperlukan masing-masing alternatif, misalnya jumlah pemanggilan *API*, beban *retrieval*, beban *reranking*, serta operasi tambahan seperti verifikasi berulang atau generasi multipel. Semakin tinggi skor pada kriteria ini berarti alternatif solusi menggunakan sumber daya yang relatif lebih hemat dan efisien untuk dijalankan pada skala eksperimen yang luas.

Fleksibilitas menilai kemampuan alternatif solusi untuk dikembangkan, diadaptasi, atau diperluas di masa depan. Solusi yang fleksibel cenderung bersifat modular, tidak terlalu terikat pada satu jenis LLM atau satu skenario tugas, serta mudah diintegrasikan dengan teknik mitigasi tambahan yang mungkin dikaji pada tugas akhir lanjutan.

Tiga alternatif solusi yang dibandingkan dalam tugas akhir ini adalah:

1. **Solusi I: Penguatan *retrieval*** (*query rewriting* + *query expansion* + *context reranking*), yang berfokus pada peningkatan kualitas kueri dan seleksi konteks sejak tahap pengambilan informasi.
2. **Solusi II: Penguatan *generation*** (*faithfulness prompting* + *attribution prompting* + *chain-of-verification*), yang mengandalkan desain *prompt* dan verifikasi berjenjang untuk menjaga kesetiaan jawaban terhadap konteks.
3. **Solusi III: Solusi tugas akhir** (*query rewriting* + *context reranking* + *active hallucination detection*), yang menggabungkan perbaikan kueri, pemilihan konteks paling relevan, serta deteksi aktif terhadap keluaran berisiko halusinatif.

Bobot kriteria yang digunakan dalam skema *weighted scoring* ditetapkan sebagai berikut: efektivitas sebesar 40%, feasibility sebesar 25%, biaya komputasi sebesar 20%, dan fleksibilitas sebesar 15%. Skor untuk masing-masing alternatif kemudian

dihitung dengan mengalikan skor *Likert* dengan bobot kriteria terkait, lalu menjumlahkannya menjadi skor total tertimbang. Ringkasan hasil penilaian disajikan pada Tabel III.4.

Tabel III.4 Kalkulasi *weighted scoring* pada alternatif solusi mitigasi *hallucination*

Kriteria	**Bobot**	**Alternatif Solusi**		
		Solusi I	**Solusi II**	**Solusi III**
Efektivitas	40%	8	7	9
Feasibility	25%	7	6	8
Biaya komputasi	20%	7	5	6
Fleksibilitas	15%	8	7	8
Skor total		**7,45**	**6,20**	**7,70**

Berdasarkan hasil *weighted scoring* pada Tabel III.4, Solusi III memperoleh skor total tertinggi dibandingkan dua alternatif lainnya. Hal ini mencerminkan kombinasi yang seimbang antara efektivitas pengurangan *hallucination*, feasibility implementasi, biaya komputasi yang masih dapat diterima, serta fleksibilitas untuk dikembangkan lebih lanjut. Oleh karena itu, tugas akhir ini memilih Solusi III, yaitu kombinasi *query rewriting*, *context reranking*, dan *active hallucination detection*, sebagai pendekatan utama mitigasi *hallucination* pada sistem RAG–LLM.

Selain hasil *weighted scoring*, pemilihan Solusi III juga didukung oleh temuan penelitian yang menunjukkan bahwa penggabungan beberapa teknik mitigasi dalam satu alur kerja dapat meningkatkan performa RAG–LLM secara nyata. Salah satu pendekatan adalah RAG-HAT (*RAG–Hallucination-Aware Tuning*), yang mengintegrasikan deteksi *hallucination*, perbaikan jawaban, dan penyesuaian model melalui *preference optimization*. Penelitian ini melaporkan penurunan *hallucination* hingga 27-44% dibandingkan RAG standar (**song2024raghat**), menunjukkan bahwa pendekatan multi-tahap efektif meningkatkan performa RAG.

Pendekatan lain yang relevan adalah MEGA-RAG, yang menggabungkan *dense retrieval*, *cross-encoder reranking*, dan penggabungan bukti sebelum proses generasi. Hasilnya, akurasi sistem meningkat sekitar 8-14% pada berbagai tugas *knowledge-intensive QA* dibandingkan RAG tanpa mekanisme gabungan (**li2023megarag**). Temuan ini memperlihatkan bahwa penggabungan teknik memberikan dampak langsung pada peningkatan kualitas jawaban.

Berdasarkan kedua studi tersebut, terlihat jelas bahwa pendekatan yang mengga-

bungkan beberapa teknik mitigasi pada tahap *retrieval* dan *post-generation* lebih mampu meningkatkan akurasi dan mengurangi *hallucination* secara konsisten. Oleh karena itu, pemilihan Solusi III dalam tugas akhir ini memiliki landasan yang kuat, karena menggabungkan *query rewriting*, *context reranking*, dan *active hallucination detection* yang sejalan dengan arah penelitian terbaru.

BAB IV

DESAIN SOLUSI

Bab ini menjelaskan dua fase *Cross-Industry Standard Process for Data Mining* (CRISP-DM) yang mencakup *data preparation* dan *modeling*. Pada fase *data preparation*, dilakukan pemilihan fitur dan prapemrosesan terhadap dataset PubMedQA yang digunakan sebagai korpus pengujian. Pada fase *modeling*, dirancang dan diimplementasikan lima konfigurasi sistem *retrieval-augmented generation* (RAG) yang mencakup konfigurasi *baseline* serta empat konfigurasi dengan teknik mitigasi halusinasi yang bervariasi. Hasil pelaksanaan setiap fase disajikan pada bagian IV.2.

IV.1 Tahapan Desain

IV.1.1 Persiapan Data (*Data Preparation*)

Tahap persiapan data bertujuan untuk memilih, memahami, dan menyiapkan dataset yang digunakan dalam proses eksperimen. Sebagaimana telah dijelaskan pada Bab III, dataset yang dipilih adalah PubMedQA, yakni dataset berbasis abstrak ilmiah dalam basis data PubMed yang memiliki struktur pertanyaan, konteks, dan jawaban yang terstandarisasi. Dari keseluruhan subset yang tersedia, digunakan subset *pqa_labeled* yang terdiri dari sekitar seribu entri berlabel pakar (*expert-labeled*).

Setiap entri dalam PubMedQA memiliki empat fitur utama yang dimanfaatkan dalam penelitian ini. Fitur pertama adalah *question*, yakni pertanyaan riset yang dirumuskan berdasarkan judul artikel ilmiah di PubMed. Fitur kedua adalah *context*, yakni daftar paragraf abstrak dari artikel ilmiah yang menjadi dasar jawaban. Fitur ketiga adalah *long_answer*, yakni narasi panjang yang merangkum temuan penelitian. Fitur keempat adalah *final_decision*, yakni label klasifikasi berupa satu dari tiga nilai, yaitu *yes*, *no*, atau *maybe*, yang diberikan oleh pakar domain sebagai jawaban ringkas atas pertanyaan.

Untuk keperluan pengindeksan dan pengambilan dokumen, paragraf-paragraf dalam fitur *context* setiap entri digabungkan menjadi satu unit teks yang mewakili satu dokumen. Dokumen yang terbentuk kemudian ditokenisasi menggunakan pendekatan

berbasis tanda baca dan spasi untuk membangun indeks *Best Match 25* (BM25). Fitur *final_decision* digunakan sebagai label referensi (*ground truth*) dalam proses evaluasi, sedangkan fitur *long_answer* digunakan sebagai konteks referensi untuk perhitungan metrik RAGAS.

IV.1.2 Pemodelan (*Modeling*)

Tahap pemodelan mencakup perancangan arsitektur sistem RAG beserta seluruh komponen mitigasi halusinasi yang akan diuji. Pada tahap ini dirumuskan lima konfigurasi sistem yang berbeda, rancangan *pipeline* RAG *baseline*, serta rancangan setiap komponen teknik mitigasi halusinasi.

IV.1.2.1 Konfigurasi Sistem

Untuk mengukur kontribusi masing-masing teknik mitigasi halusinasi secara terpisah maupun gabungan, dirancang lima konfigurasi sistem yang bervariasi. Konfigurasi pertama adalah *baseline*, yakni sistem RAG tanpa teknik mitigasi apa pun yang berfungsi sebagai titik acuan pengukuran. Konfigurasi kedua menambahkan komponen *query rewriting* pada konfigurasi *baseline*. Konfigurasi ketiga menambahkan komponen *context reranking*. Konfigurasi keempat menambahkan komponen *active hallucination detection*. Konfigurasi kelima merupakan konfigurasi gabungan yang mengaktifkan seluruh teknik mitigasi secara bersamaan. Ringkasan kelima konfigurasi tersebut ditunjukkan pada Tabel IV.1.

Tabel IV.1 Matriks Konfigurasi Sistem RAG

Konfigurasi	***Query Rewriting***	***Context Reranking***	***Active Detection***
Baseline	×	×	×
Baseline + QR	✓	×	×
Baseline + CR	×	✓	×
Baseline + AD	×	×	✓
Combined	✓	✓	✓

Selain matriks konfigurasi, ditetapkan pula parameter teknis yang berlaku pada seluruh konfigurasi secara konsisten. Parameter-parameter tersebut disajikan pada Tabel IV.2.

Tabel IV.2 Parameter Teknis Sistem RAG

Komponen / Parameter	**Nilai**
Model LLM	[TODO: nama model, mis. llama3.1:8b]
Framework inferensi	Ollama (lokal)
Metode *retrieval*	BM25 (*library* rank-bm25)
Jumlah dokumen awal (*top-k retrieval*)	10
Jumlah dokumen setelah *reranking* (*top-k final*)	3
Model *reranker*	[TODO: nama CrossEncoder, mis. cross-encoder/ms-marco-MiniLM-L-6-v2]
Model *embedding* (RAGAS)	[TODO: nama model embedding]
Temperature LLM	0
Jumlah sampel evaluasi	1.000 entri
Framework evaluasi	RAGAS 0.4.3

IV.1.2.2 Desain *Pipeline* RAG *Baseline*

Pipeline RAG *baseline* dirancang sebagai fondasi sistem yang akan dibandingkan dengan konfigurasi-konfigurasi berikutnya. *Pipeline* ini terdiri dari tiga tahapan utama yang dijalankan secara berurutan, yakni *indexing pipeline*, *retrieval pipeline*, dan *generation pipeline*, sebagaimana diilustrasikan pada Gambar **??**.

Gambar IV.1 Diagram Aliran *Pipeline* RAG *Baseline*

Tiga tahapan dalam *pipeline* RAG *baseline* dirancang sebagai berikut.

1. *Indexing Pipeline*
   Tahap *indexing* bertujuan membangun representasi teks korpus yang memungkinkan pencarian berbasis relevansi. Pada penelitian ini, *indexing* dilakukan menggunakan metode BM25 yang bekerja pada level token. Proses ini terdiri dari dua langkah:
   a. Tokenisasi korpus: setiap dokumen dalam korpus PubMedQA ditokenisasi dengan memisahkan teks berdasarkan karakter spasi dan tanda baca. Teks diubah menjadi huruf kecil sebelum tokenisasi untuk mengurangi sensitivitas terhadap kapitalisasi.
   b. Pembangunan indeks BM25: token-token hasil tokenisasi dari seluruh dokumen korpus digunakan untuk membangun objek indeks *BM25Okapi*.

Indeks ini menyimpan statistik frekuensi token yang diperlukan untuk menghitung skor relevansi pada saat *retrieval*.

2. *Retrieval Pipeline*

   Tahap *retrieval* bertujuan mengambil dokumen yang paling relevan dari korpus berdasarkan pertanyaan masukan. Proses ini terdiri dari tiga langkah:

   a. Tokenisasi kueri: pertanyaan masukan ditokenisasi menggunakan prosedur yang sama dengan tokenisasi korpus sehingga representasinya konsisten dengan indeks BM25.

   b. Perhitungan skor BM25: skor relevansi dihitung untuk setiap dokumen dalam korpus menggunakan fungsi *get_scores* pada indeks BM25. Skor ini mencerminkan tingkat kesesuaian antara token kueri dan token dokumen, mempertimbangkan frekuensi kemunculan dan panjang dokumen.

   c. Seleksi *top-k*: dokumen-dokumen diurutkan berdasarkan skor BM25 dari tertinggi ke terendah, kemudian diambil 10 dokumen teratas sebagai konteks kandidat yang akan diteruskan ke tahap generasi.

3. *Generation Pipeline*

   Tahap *generation* bertujuan menghasilkan jawaban berdasarkan pertanyaan dan konteks yang telah diambil. Proses ini terdiri dari dua langkah:

   a. Perumusan *prompt*: tiga dokumen teratas dari hasil *retrieval* digabungkan sebagai konteks dalam *prompt* yang diberikan kepada LLM. Selain konteks, *prompt* juga memuat instruksi agar LLM menjawab dalam format *yes*, *no*, atau *maybe* pada baris terakhir keluarannya. Pembatasan format ini diperlukan agar keluaran LLM dapat diperbandingkan secara langsung dengan label referensi *final_decision* pada dataset PubMedQA.

   b. Penghasilan jawaban: LLM menerima *prompt* yang telah dirumuskan dan menghasilkan respons. Dari respons tersebut, label prediksi diekstraksi menggunakan strategi tiga lapis, yaitu pencocokan baris terakhir, penelusuran kata kunci dalam seluruh teks, dan ekstraksi berbasis ekspresi reguler.

IV.1.2.3 Desain Komponen Mitigasi Halusinasi

Tiga komponen mitigasi halusinasi dirancang untuk diintegrasikan ke dalam *pipeline* RAG *baseline* pada konfigurasi yang relevan. Setiap komponen bekerja pada tahap yang berbeda dalam *pipeline*, sehingga bersifat saling melengkapi.

1. *Query Rewriting*

   Komponen *query rewriting* bertujuan memperjelas dan memperbaiki struktur pertanyaan pengguna sebelum proses pencarian dokumen dilakukan. Perta-

nyaan asli pada dataset biomedis seperti PubMedQA sering kali mengandung istilah teknis yang ambigu atau kurang eksplisit dalam menyatakan kebutuhan informasi. Perbaikan kueri diharapkan meningkatkan presisi dokumen yang diambil oleh BM25, sehingga konteks yang disediakan untuk LLM lebih relevan.

Komponen ini diintegrasikan di awal *pipeline*, sebelum tahap *retrieval*. Proses *query rewriting* dirancang sebagai berikut:

a. Perumusan *prompt* penulisan ulang: pertanyaan asli diberikan kepada LLM bersama instruksi untuk menghasilkan versi pertanyaan yang lebih jelas, lebih eksplisit dalam menyatakan fokus informasi yang dicari, dan lebih sesuai dengan terminologi ilmiah biomedis.
b. Penggantian kueri: pertanyaan hasil penulisan ulang menggantikan pertanyaan asli pada seluruh tahap *retrieval* berikutnya. Pertanyaan asli tetap disimpan dan digunakan pada tahap evaluasi sebagai konteks penilaian relevansi jawaban.

Aliran proses *pipeline* dengan *query rewriting* diilustrasikan pada Gambar **??**.

Gambar IV.2 Diagram Aliran *Pipeline* RAG dengan *Query Rewriting*

2. *Context Reranking*

Komponen *context reranking* bertujuan memilih ulang urutan dokumen yang telah diambil oleh BM25 berdasarkan relevansi semantik yang lebih presisi. BM25 mengandalkan kecocokan token yang bersifat leksikal, sehingga dokumen yang relevan secara makna namun menggunakan terminologi berbeda dapat memperoleh skor rendah. Sebaliknya, dokumen yang mengandung banyak token yang sama dengan kueri tetapi tidak relevan secara kontekstual dapat memperoleh skor tinggi. Mekanisme *reranking* berbasis *cross-encoder* dapat mengoreksi ketidaktepatan ini.

Komponen ini diintegrasikan antara tahap *retrieval* dan tahap *generation*. Proses *context reranking* dirancang sebagai berikut:

a. Pemberian skor *cross-encoder*: sepuluh dokumen teratas hasil BM25 dipasangkan satu per satu dengan kueri, kemudian masing-masing pasangan diberi skor relevansi oleh model *cross-encoder*. Berbeda dengan *bi-encoder*, model *cross-encoder* memproses pasangan kueri dan dokumen secara bersamaan sehingga menghasilkan penilaian relevansi yang lebih akurat.
b. Pengurutan ulang dan seleksi: dokumen-dokumen diurutkan ulang berdasarkan skor *cross-encoder* dari tertinggi ke terendah. Tiga dokumen dengan skor tertinggi dipilih sebagai konteks akhir yang diteruskan ke

tahap *generation*.
Aliran proses *pipeline* dengan *context reranking* diilustrasikan pada Gambar **??**.

Gambar IV.3 Diagram Aliran *Pipeline* RAG dengan *Context Reranking*

3. *Active Hallucination Detection*
   Komponen *active hallucination detection* bertujuan memeriksa kesesuaian antara jawaban yang dihasilkan LLM dengan konteks yang tersedia sebelum jawaban tersebut dianggap final. Deteksi dilakukan secara aktif, artinya proses verifikasi berjalan setelah jawaban dihasilkan tetapi sebelum hasil evaluasi dicatat. Rancangan detail komponen ini akan dijelaskan lebih lanjut setelah pengujian awal komponen *query rewriting* dan *context reranking* selesai dilakukan.
   Aliran proses *pipeline* dengan *active hallucination detection* diilustrasikan pada Gambar **??**.

Gambar IV.4 Diagram Aliran *Pipeline* RAG dengan *Active Hallucination Detection*

IV.2 Hasil Desain

Bagian ini menyajikan hasil nyata dari setiap tahapan desain yang telah dijelaskan pada bagian IV.1. Hasil mencakup kondisi dataset setelah prapemrosesan, keluaran konkret dari setiap komponen *pipeline*, serta verifikasi bahwa seluruh konfigurasi sistem dapat berjalan dengan benar.

IV.2.1 Hasil Persiapan Data

Dataset PubMedQA subset *pqa_labeled* dimuat dari repositori Hugging Face menggunakan identitas dataset `qiaojin/PubMedQA` dengan konfigurasi `pqa_labeled`. Statistik distribusi dataset disajikan pada Tabel IV.3.

Tabel IV.3 Statistik Dataset PubMedQA *pqa_labeled*

Keterangan	**Nilai**
Total entri	1.000
Entri berlabel *yes*	[TODO: isi setelah eksplorasi data]
Entri berlabel *no*	[TODO: isi setelah eksplorasi data]
Entri berlabel *maybe*	[TODO: isi setelah eksplorasi data]
Rata-rata jumlah paragraf konteks	[TODO: isi setelah eksplorasi data]
Rata-rata panjang teks dokumen (token)	[TODO: isi setelah eksplorasi data]
Jumlah total dokumen dalam indeks BM25	1.000

Tabel IV.4 menunjukkan contoh satu entri dari dataset PubMedQA setelah fitur yang diperlukan diekstraksi dan dokumen disatukan.

Tabel IV.4 Contoh Entri Dataset PubMedQA Setelah Prapemrosesan

Fitur	**Nilai**
question	[TODO: salin satu pertanyaan contoh dari dataset]
context (dokumen gabungan)	[TODO: salin beberapa kalimat awal dari konteks gabungan]
final_decision	[TODO: yes / no / maybe]

IV.2.2 Hasil Pemodelan

IV.2.2.1 Hasil *Pipeline* RAG *Baseline*

Pipeline RAG *baseline* diimplementasikan dan diuji menggunakan sampel pertanyaan dari dataset PubMedQA. Proses *retrieval* menggunakan BM25 berhasil mengambil dokumen yang relevan berdasarkan kesesuaian token, dan tahap *generation* menghasilkan jawaban dalam format *yes*, *no*, atau *maybe* sesuai instruksi *prompt*. Tabel IV.5 menunjukkan contoh keluaran *pipeline baseline* untuk satu pertanyaan sampel.

Tabel IV.5 Contoh Keluaran *Pipeline* RAG *Baseline*

Komponen	**Keluaran**
Query masukan	[TODO: salin pertanyaan contoh]
Dokumen terambil (ringkasan)	[TODO: ringkasan singkat 3 dokumen teratas dari BM25]
Jawaban LLM	[TODO: salin jawaban yang dihasilkan LLM]
Label prediksi	[TODO: yes / no / maybe]
Label referensi	[TODO: label ground truth dari dataset]

IV.2.2.2 Hasil Komponen Mitigasi Halusinasi

Setiap komponen mitigasi halusinasi diuji secara terpisah untuk memverifikasi bahwa integrasinya ke dalam *pipeline* berjalan dengan benar sebelum evaluasi skala penuh dilakukan.

1. Hasil *Query Rewriting*
   Komponen *query rewriting* diuji dengan memberikan pertanyaan dari dataset PubMedQA kepada LLM untuk ditulis ulang. Gambar **??** menunjukkan perbandingan antara pertanyaan asli dan pertanyaan hasil penulisan ulang pada

beberapa sampel.

Gambar IV.5 Contoh Perbandingan Pertanyaan Asli dan Hasil *Query Rewriting*

Pertanyaan yang dihasilkan menunjukkan perbaikan dalam hal kejelasan terminologi dan eksplisitas fokus informasi yang dicari. Perubahan ini diharapkan meningkatkan skor BM25 untuk dokumen-dokumen yang relevan secara semantik.

2. Hasil *Context Reranking*
   Komponen *context reranking* diuji dengan memberikan sepuluh dokumen teratas hasil BM25 kepada model *cross-encoder* untuk diberi skor ulang. Gambar **??** menunjukkan perbandingan urutan dokumen sebelum dan sesudah proses *reranking* untuk satu pertanyaan sampel.

Gambar IV.6 Contoh Perubahan Urutan Dokumen Sebelum dan Sesudah *Context Reranking*

Perubahan urutan yang terjadi menunjukkan bahwa model *cross-encoder* berhasil mengidentifikasi dokumen yang lebih relevan secara semantik dibandingkan pemeringkatan awal BM25, terutama pada kasus di mana dokumen relevan menggunakan terminologi yang berbeda dari kueri.

3. Hasil *Active Hallucination Detection*
   Pengujian komponen *active hallucination detection* akan dilaporkan setelah implementasi komponen ini selesai. Hasil pengujian awal akan disajikan pada bagian ini bersama contoh kasus deteksi yang berhasil dan tidak berhasil.

IV.2.2.3 Hasil Implementasi *Pipeline* Terpadu

Setelah seluruh komponen mitigasi halusinasi berhasil diimplementasikan dan diverifikasi secara terpisah, dilakukan pengujian integrasi pada konfigurasi *combined* yang mengaktifkan ketiga komponen sekaligus. Hasil pengujian awal ditunjukkan pada Tabel IV.6.

Tabel IV.6 Contoh Perbandingan Keluaran Lima Konfigurasi untuk Satu Pertanyaan Sampel

Konfigurasi	**Ringkasan Jawaban LLM**	**Label Prediksi**	**Label Referensi**
Baseline	[TODO]	[TODO]	[TODO]
Baseline + QR	[TODO]	[TODO]	[TODO]
Baseline + CR	[TODO]	[TODO]	[TODO]
Baseline + AD	[TODO]	[TODO]	[TODO]
Combined	[TODO]	[TODO]	[TODO]

BAB V

IMPLEMENTASI

Bab ini menjelaskan implementasi teknis dari desain solusi yang telah dirumuskan pada Bab IV. Implementasi mencakup penyiapan lingkungan pengembangan, pembangunan komponen *pipeline* RAG *baseline*, integrasi tiga komponen mitigasi halusinasi, serta penyiapan *framework* evaluasi. Seluruh implementasi dilakukan dalam bentuk *notebook* Python yang dijalankan secara lokal.

V.1 Lingkungan Implementasi

V.1.1 Perangkat Keras

Seluruh proses implementasi dan eksperimen dijalankan pada satu unit komputer dengan spesifikasi yang disajikan pada Tabel V.1.

Tabel V.1 Spesifikasi Perangkat Keras

Komponen	**Spesifikasi**
Prosesor	[TODO: mis. Intel Core i7-12700H]
Memori RAM	[TODO: mis. 16 GB DDR5]
GPU	[TODO: mis. NVIDIA RTX 3060 6 GB / CPU only]
Penyimpanan	[TODO: mis. 512 GB NVMe SSD]
Sistem Operasi	[TODO: mis. Windows 11 / Ubuntu 22.04]

V.1.2 Perangkat Lunak dan Dependensi

Implementasi dilakukan menggunakan Python 3.11 dalam lingkungan virtual yang terisolasi. Model LLM dijalankan secara lokal menggunakan Ollama, sehingga tidak diperlukan akses internet atau *API key* pihak ketiga pada saat inferensi. Daftar pustaka (*library*) utama yang digunakan disajikan pada Tabel V.2.

Tabel V.2 Daftar Dependensi Utama

Library	**Versi**	**Fungsi**
`rank-bm25`	[TODO]	Pembangunan dan pencarian indeks BM
`datasets`	[TODO]	Pemuatan dataset PubMedQA dari Hugg
`langchain-ollama`	[TODO]	Antarmuka LLM melalui Ollama
`sentence-transformers`	[TODO]	Model *cross-encoder* untuk *reranking*
`ragas`	0.4.3	*Framework* evaluasi RAG
`numpy`	[TODO]	Operasi numerik
`pandas`	[TODO]	Pemrosesan dan penyimpanan hasil eval
`tqdm`	[TODO]	Tampilan progres eksperimen
Ollama	[TODO]	*Runtime* LLM lokal
Model LLM	[TODO: mis. llama3.1:8b]	Model bahasa untuk generasi jawaban
Model *embedding*	[TODO: mis. nomic-embed-text]	*Embedding* untuk evaluasi RAGAS

Struktur *notebook* eksperimen diorganisasikan menjadi lima berkas terpisah yang bersesuaian dengan lima konfigurasi sistem, sebagaimana ditunjukkan pada Tabel V.3.

Tabel V.3 Struktur *Notebook* Eksperimen

Berkas *Notebook*	**Konfigurasi**
`03-RAG-Baseline-Evaluation.ipynb`	*Baseline*
`04-RAG-QueryRewriting-Evaluation.ipynb`	*Baseline* + QR
`05-RAG-ContextReranking-Evaluation.ipynb`	*Baseline* + CR
`06-RAG-ActiveDetection-Evaluation.ipynb`	*Baseline* + AD
`07-RAG-Combined-Evaluation.ipynb`	*Combined*

V.2 Implementasi Persiapan Data

Dataset PubMedQA dimuat menggunakan pustaka `datasets` dari Hugging Face dengan identitas dataset `qiaojin/PubMedQA` dan konfigurasi `pqa_labeled`. Setiap entri dataset yang dimuat memiliki struktur berupa *dictionary* dengan kunci *question*, *context*, *long_answer*, dan *final_decision*.

Untuk keperluan pengindeksan BM25, fitur *context* setiap entri yang berisi daftar paragraf digabungkan menjadi satu unit teks menggunakan pemisah spasi. Teks gabungan tersebut kemudian ditokenisasi dengan memisahkan karakter berdasarkan tanda baca dan spasi, serta diubah seluruhnya ke dalam huruf kecil. Hasil tokenisasi seluruh entri membentuk korpus bertokenisasi yang digunakan sebagai masukan

untuk membangun objek indeks *BM25Okapi*.

Untuk efisiensi eksperimen, indeks BM25 yang telah dibangun disimpan ke dalam berkas `pubmedqa_bm25.pkl` menggunakan pustaka `pickle`. Pada eksekusi berikutnya, indeks dapat dimuat kembali tanpa perlu membangun ulang dari awal.

V.3 Implementasi *Pipeline* RAG *Baseline*

V.3.1 Implementasi *Retrieval Pipeline*

Fungsi *retrieval* menggunakan indeks BM25 yang telah dibangun sebelumnya. Pertanyaan masukan ditokenisasi menggunakan prosedur yang sama dengan tokenisasi korpus, kemudian skor relevansi dihitung terhadap seluruh dokumen menggunakan metode *get_scores*. Dokumen-dokumen diurutkan berdasarkan skor dari tertinggi ke terendah menggunakan *argsort*, dan sepuluh dokumen dengan skor tertinggi dipilih sebagai konteks kandidat.

V.3.2 Implementasi *Generation Pipeline*

Tahap generasi menggunakan model LLM yang diakses melalui antarmuka `langchain-ollama`. *Prompt* yang dikirimkan ke LLM memiliki struktur sebagai berikut: diawali dengan instruksi peran sebagai asisten peneliti medis, diikuti oleh tiga dokumen konteks yang dipisahkan oleh baris pemisah, kemudian pertanyaan masukan, dan diakhiri dengan instruksi eksplisit agar LLM menuliskan salah satu dari tiga kata *yes*, *no*, atau *maybe* pada baris terakhir keluarannya.

Keluaran LLM berupa teks bebas yang kemudian diproses oleh fungsi ekstraksi label. Fungsi ini mencoba menentukan label prediksi melalui tiga strategi secara berurutan. Strategi pertama adalah pencocokan langsung pada baris terakhir keluaran. Strategi kedua adalah penelusuran kata kunci *yes*, *no*, atau *maybe* dalam seluruh teks keluaran. Strategi ketiga adalah pencocokan berbasis ekspresi reguler sebagai penanganan kasus tepi (*edge case*). Apabila ketiga strategi tidak menghasilkan pencocokan, label ditetapkan sebagai *unknown*.

V.3.3 Implementasi Penyimpanan Hasil Inkremental

Mengingat waktu eksekusi yang panjang untuk seribu entri, proses evaluasi dibagi menjadi dua fase yang masing-masing menyimpan hasilnya secara inkremental. Fase pertama menjalankan seluruh proses *retrieval* dan *generation* serta menyimpan hasilnya ke dalam berkas `baseline_phase1_answers.json`. Fase kedua menjalankan evaluasi RAGAS dan menyimpan hasilnya ke dalam berkas `baseline_phase2_ragas.json`. Kedua berkas diperiksa keberadaannya pada awal eksekusi sehingga proses dapat dilanjutkan dari titik terakhir apabila terhenti sebelum selesai.

V.4 Implementasi Komponen Mitigasi Halusinasi

V.4.1 Implementasi *Query Rewriting*

Komponen *query rewriting* diimplementasikan sebagai fungsi yang menerima pertanyaan asli dan mengembalikan versi yang telah ditulis ulang. Fungsi ini mengirimkan *prompt* khusus kepada LLM yang menginstruksikan model untuk menghasilkan versi pertanyaan yang lebih jelas, lebih eksplisit dalam terminologi biomedis, dan lebih tepat dalam menggambarkan kebutuhan informasi. LLM yang digunakan untuk *query rewriting* adalah model yang sama dengan model generasi jawaban, yakni [TODO: nama model].

Hasil *query rewriting* menggantikan pertanyaan asli pada seluruh proses *retrieval*. Pertanyaan asli tetap disimpan dan diteruskan ke tahap *generation* serta evaluasi RAGAS agar jawaban yang dihasilkan tetap relevan terhadap pertanyaan asal pengguna.

V.4.2 Implementasi *Context Reranking*

Komponen *context reranking* diimplementasikan menggunakan model *cross-encoder* dari pustaka `sentence-transformers`. Sepuluh dokumen teratas hasil BM25 dipasangkan satu per satu dengan kueri, menghasilkan sepuluh pasangan masukan. Setiap pasangan diberi skor oleh model *cross-encoder* menggunakan metode *predict*. Dokumen kemudian diurutkan ulang berdasarkan skor tersebut dan tiga dokumen teratas dipilih sebagai konteks akhir yang diteruskan ke tahap generasi.

Model *cross-encoder* yang digunakan adalah [TODO: nama model, mis. `cross-encoder/ms-marco-Mini` yang memiliki keseimbangan yang baik antara akurasi penilaian relevansi dan kecepatan inferensi. Model dijalankan secara lokal tanpa memerlukan koneksi internet pada saat eksperimen berlangsung.

V.4.3 Implementasi *Active Hallucination Detection*

Implementasi komponen *active hallucination detection* akan dijelaskan pada bagian ini setelah proses perancangan dan pengujian awal komponen tersebut selesai dilakukan.

V.5 Implementasi *Framework* Evaluasi

Evaluasi kuantitatif menggunakan *framework* RAGAS versi 0.4.3 yang diintegrasikan dengan model LLM dan *embedding* lokal melalui Ollama. Empat metrik RAGAS digunakan, yakni *faithfulness*, *answer relevancy*, *context precision*, dan *context recall*. Metrik-metrik tersebut diakses menggunakan antarmuka lama RAGAS

(*underscore-prefixed metrics*) yang kompatibel dengan `LangchainLLMWrapper` dari Ollama.

Untuk mengatasi keterbatasan kapasitas inferensi Ollama yang memproses satu permintaan pada satu waktu, konfigurasi `RunConfig` RAGAS ditetapkan dengan nilai `max_workers` sebesar satu dan nilai `timeout` sebesar 300 detik. Pengaturan ini mencegah kondisi *timeout* yang terjadi apabila RAGAS mengirimkan permintaan secara paralel ke Ollama.

Hasil evaluasi RAGAS setiap sampel disimpan secara inkremental ke dalam berkas JSON sehingga proses evaluasi dapat dilanjutkan dari titik terakhir tanpa kehilangan data apabila terjadi gangguan. Setelah seluruh sampel selesai dievaluasi, hasil digabungkan menjadi satu berkas CSV yang memuat seluruh metrik per sampel beserta label prediksi dan label referensi.

BAB VI

EVALUASI

Bab ini menyajikan metodologi, hasil, dan pembahasan evaluasi terhadap lima konfigurasi sistem RAG yang telah diimplementasikan pada Bab V. Evaluasi dilakukan untuk mengukur sejauh mana setiap teknik mitigasi halusinasi berkontribusi terhadap peningkatan kualitas jawaban yang dihasilkan sistem. Penilaian mencakup dua dimensi utama, yakni kualitas jawaban berbasis metrik RAGAS dan ketepatan klasifikasi label menggunakan label referensi dari dataset PubMedQA.

VI.1 Metodologi Evaluasi

VI.1.1 Metrik Evaluasi RAGAS

Evaluasi kualitas sistem RAG dilakukan menggunakan empat metrik dari *framework Retrieval Augmented Generation Assessment* (RAGAS) (**es2023ragas**). Keempat metrik ini mengukur aspek yang berbeda dari kualitas *retrieval* dan *generation*, sehingga memberikan gambaran yang komprehensif terhadap kinerja sistem.

1. *Faithfulness*

   Faithfulness mengukur konsistensi faktual antara jawaban yang dihasilkan LLM dan konteks yang diberikan. Metrik ini menilai sejauh mana setiap pernyataan dalam jawaban dapat didukung oleh bukti dalam konteks. Semakin tinggi nilai *faithfulness*, semakin kecil peluang jawaban mengandung informasi yang tidak ada dalam konteks atau bertentangan dengan konteks. Metrik ini dihitung dengan persamaan berikut.

   $$F = \frac{|\text{pernyataan dalam jawaban yang didukung konteks}|}{|\text{jumlah total pernyataan dalam jawaban}|} \tag{VI.1}$$

2. *Answer Relevancy*

   Answer relevancy mengukur seberapa relevan jawaban yang dihasilkan terhadap pertanyaan masukan. RAGAS menghitung metrik ini dengan cara meminta LLM menghasilkan beberapa pertanyaan hipotetis berdasarkan jawaban yang ada, kemudian mengukur kesamaan semantik antara pertanyaan hipo-

tetis tersebut dengan pertanyaan asli menggunakan *cosine similarity*. Metrik ini dihitung dengan persamaan berikut.

$$AR = \frac{1}{n}\sum_{i=1}^{n} \text{sim}(q, q_i) \tag{VI.2}$$

Keterangan:
q = pertanyaan asli
q_i = pertanyaan hipotetis yang dihasilkan LLM berdasarkan jawaban
n = jumlah pertanyaan hipotetis yang dihasilkan

3. *Context Precision*
   Context precision mengukur proporsi dokumen yang relevan di antara seluruh dokumen yang diambil oleh *retriever*. Metrik ini menilai kemampuan sistem dalam menghindari pengambilan dokumen yang tidak relevan (*false positive*). Relevansi setiap dokumen ditentukan oleh LLM berdasarkan perbandingannya dengan jawaban referensi (*ground truth*). Metrik ini dihitung dengan persamaan berikut.

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K}(\text{Precision@}k \times v_k)}{\text{jumlah item relevan dalam K teratas}} \tag{VI.3}$$

Keterangan:
K = jumlah dokumen yang diambil
$v_k \in \{0, 1\}$ = indikator relevansi pada dokumen ke-k

4. *Context Recall*
   Context recall mengukur seberapa banyak informasi dari jawaban referensi yang berhasil ditemukan dalam konteks yang diambil. LLM menguraikan jawaban referensi menjadi sejumlah klaim faktual, kemudian menghitung proporsi klaim yang dapat dikaitkan dengan salah satu dokumen dalam konteks. Metrik ini dihitung dengan persamaan berikut.

$$\text{Context Recall} = \frac{|\text{klaim dalam referensi yang didukung konteks}|}{|\text{jumlah total klaim dalam referensi}|} \tag{VI.4}$$

VI.1.2 Metrik Evaluasi Tambahan

Selain metrik RAGAS, digunakan dua metrik tambahan yang memanfaatkan label referensi *final_decision* dari dataset PubMedQA.

1. *Label Accuracy*
   Label accuracy mengukur proporsi jawaban LLM yang label prediksinya (*yes*, *no*, atau *maybe*) sesuai dengan label referensi dalam dataset. Metrik ini mengukur kemampuan sistem secara keseluruhan dalam menjawab pertanyaan biomedis secara tepat. Metrik ini dihitung dengan persamaan berikut.

$$\text{Accuracy} = \frac{|\text{prediksi yang sesuai label referensi}|}{|\text{jumlah total sampel}|} \tag{VI.5}$$

2. *Hallucination Rate*
   Hallucination rate didefinisikan sebagai proporsi sampel di mana jawaban LLM tidak dapat dicocokkan dengan label mana pun (*unknown*), baik karena format keluaran tidak sesuai instruksi maupun karena LLM menghasilkan respons di luar tiga pilihan yang ditetapkan. Metrik ini dihitung dengan persamaan berikut.

$$\text{Hallucination Rate} = \frac{|\text{prediksi bernilai } \textit{unknown}|}{|\text{jumlah total sampel}|} \tag{VI.6}$$

VI.1.3 Prosedur Evaluasi

Evaluasi dilakukan terhadap 1.000 entri dari subset *pqa_labeled* PubMedQA secara berurutan. Setiap konfigurasi dijalankan secara independen dengan parameter LLM yang identik, yakni *temperature* bernilai nol untuk memastikan keluaran yang deterministik dan dapat direproduksi. Untuk setiap entri, dicatat enam nilai, yaitu *faithfulness*, *answer relevancy*, *context precision*, *context recall*, label prediksi, dan label referensi. Agregasi akhir dihitung sebagai rata-rata aritmetika dari masing-masing metrik di seluruh sampel yang berhasil dievaluasi.

VI.2 Hasil Evaluasi

VI.2.1 Hasil Konfigurasi *Baseline*

Konfigurasi *baseline* dijalankan menggunakan BM25 sebagai *retriever* tanpa teknik mitigasi apa pun. Ringkasan hasil evaluasi konfigurasi *baseline* disajikan pada Tabel VI.1.

Tabel VI.1 Ringkasan Hasil Evaluasi Konfigurasi *Baseline*

Metrik	**Nilai**
Faithfulness	[TODO]
Answer Relevancy	[TODO]
Context Precision	[TODO]
Context Recall	[TODO]
Label Accuracy	[TODO]
Hallucination Rate	[TODO]

VI.2.2 Hasil Konfigurasi *Baseline* + *Query Rewriting*

Konfigurasi ini menambahkan komponen *query rewriting* pada konfigurasi *baseline*. Ringkasan hasil evaluasi disajikan pada Tabel VI.2.

Tabel VI.2 Ringkasan Hasil Evaluasi Konfigurasi *Baseline* + *Query Rewriting*

Metrik	**Nilai**
Faithfulness	[TODO]
Answer Relevancy	[TODO]
Context Precision	[TODO]
Context Recall	[TODO]
Label Accuracy	[TODO]
Hallucination Rate	[TODO]

VI.2.3 Hasil Konfigurasi *Baseline* + *Context Reranking*

Konfigurasi ini menambahkan komponen *context reranking* menggunakan model *cross-encoder* pada konfigurasi *baseline*. Ringkasan hasil evaluasi disajikan pada Tabel VI.3.

Tabel VI.3 Ringkasan Hasil Evaluasi Konfigurasi *Baseline* + *Context Reranking*

Metrik	**Nilai**
Faithfulness	[TODO]
Answer Relevancy	[TODO]
Context Precision	[TODO]
Context Recall	[TODO]
Label Accuracy	[TODO]
Hallucination Rate	[TODO]

VI.2.4 Hasil Konfigurasi *Baseline + Active Hallucination Detection*

Konfigurasi ini menambahkan komponen *active hallucination detection* pada konfigurasi *baseline*. Ringkasan hasil evaluasi disajikan pada Tabel VI.4.

Tabel VI.4 Ringkasan Hasil Evaluasi Konfigurasi *Baseline + Active Hallucination Detection*

Metrik	**Nilai**
Faithfulness	[TODO]
Answer Relevancy	[TODO]
Context Precision	[TODO]
Context Recall	[TODO]
Label Accuracy	[TODO]
Hallucination Rate	[TODO]

VI.2.5 Hasil Konfigurasi *Combined*

Konfigurasi *combined* mengaktifkan ketiga teknik mitigasi halusinasi secara bersamaan. Ringkasan hasil evaluasi disajikan pada Tabel VI.5.

Tabel VI.5 Ringkasan Hasil Evaluasi Konfigurasi *Combined*

Metrik	**Nilai**
Faithfulness	[TODO]
Answer Relevancy	[TODO]
Context Precision	[TODO]
Context Recall	[TODO]
Label Accuracy	[TODO]
Hallucination Rate	[TODO]

VI.3 Perbandingan Antar Konfigurasi

Tabel VI.6 merangkum hasil evaluasi kelima konfigurasi secara berdampingan untuk memudahkan pembandingan kontribusi setiap teknik mitigasi.

Tabel VI.6 Perbandingan Hasil Evaluasi Seluruh Konfigurasi

Metrik	**Baseline**	**+QR**	**+CR**	**+AD**	**Combined**
Faithfulness	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Answer Relevancy	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Context Precision	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Context Recall	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Label Accuracy	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]
Hallucination Rate	[TODO]	[TODO]	[TODO]	[TODO]	[TODO]

Gambar VI.1 Perbandingan Nilai Metrik Evaluasi Seluruh Konfigurasi

VI.4 Analisis dan Pembahasan

VI.4.1 Dampak *Query Rewriting*

[TODO: Isi setelah hasil evaluasi tersedia.]

Perbandingan hasil antara konfigurasi *baseline* dan konfigurasi *baseline* + *query rewriting* menunjukkan bahwa [TODO: ringkasan temuan]. Peningkatan atau penurunan yang terjadi pada metrik *context precision* dan *context recall* mencerminkan dampak langsung dari perubahan kueri terhadap kualitas *retrieval*. [TODO: analisis lebih lanjut].

VI.4.2 Dampak *Context Reranking*

[TODO: Isi setelah hasil evaluasi tersedia.]

Konfigurasi *context reranking* mengubah urutan dokumen yang diberikan kepada LLM berdasarkan skor relevansi *cross-encoder*. Perubahan ini berdampak pada [TODO: metrik mana yang berubah dan dalam arah apa]. [TODO: analisis lebih lanjut].

VI.4.3 Dampak *Active Hallucination Detection*

[TODO: Isi setelah implementasi dan evaluasi Active Hallucination Detection selesai.]

VI.4.4 Analisis Konfigurasi *Combined*

[TODO: Isi setelah hasil evaluasi seluruh konfigurasi tersedia.]

Konfigurasi *combined* merupakan konfigurasi yang mengaktifkan seluruh teknik mitigasi secara bersamaan. Hasil yang diperoleh menunjukkan [TODO: apakah ada

sinergi atau justru konflik antar komponen]. [TODO: analisis lebih lanjut].

VI.4.5 Diskusi Umum

[TODO: Isi setelah seluruh hasil evaluasi tersedia dan dianalisis.]

BAB VII

PENUTUP

VII.1 Kesimpulan

Jelaskan secara detail langkah-langkah rencana selanjutnya, hal-hal yang diperlukan atau akan disiapkan, dan risiko dan mitigasinya, yang meliputi:

VII.2 Saran

asdfas

LAMPIRAN A

KODE PROGRAM

Pada umumnya, kode program tidak perlu disertakan di dalam laporan tugas akhir karena kode program biasanya sangat panjang dan sudah disimpan dalam bentuk berkas-berkas elektronik terpisah di repositori daring. Namun, jika ada bagian kode program singkat yang penting untuk ditulis dalam laporan tugas akhir, bagian tersebut dapat disertakan di dalam laporan.

A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik

```
% -- Example of pseudocode and source code listing --
% -- Gunakan minipage agar listing tidak terpotong ke halaman berikutnya --



\begin{minipage}{\textwidth}
\begin{lstlisting}[caption={Iterative binary search in Python}, label={lst:binary_iter}, frame=single, captionpos=t, language=Python]
def binary_search(arr, target):
    """
    Performs binary search on a sorted array.

    Args:
        arr: A sorted list of elements
        target: The element to search for

    Returns:
        Index of target if found, -1 otherwise
    """
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

```

```
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1


\end{lstlisting}
\end{minipage}
```

Listing A.1 Binary search code

A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak

asdjfkjashdkjfas

LAMPIRAN B

HASIL SURVEI LAPANGAN

B.1 Foto Survei Lokasi 1

Teks survei lokasi 1.

B.2 Foto Survei Lokasi 2

Teks survei lokasi 2.

B.3 Transkrip Wawancara dengan Narasumber

Teks transkrip wawancara.